Digital Engineering 10 (2026) 100099

Contents lists available at ScienceDirect

Digital Engineering

journal homepage: www.sciencedirect.com/journal/digital-engineering

Full Length Article

Self-evolving digital twins in flexible manufacturing: A large language model-powered multi-agent approach

Ding Gao, Xiaojian Wen, Jianhao Lv, Jiahui Si, Cong Wang, Jinsong Bao *

College of Mechanical Engineering, Donghua University, Shanghai, 201620 China

ARTICLE INFO

Keywords:
Digital twin
Large language model
Multi-agent system
Flexible production line
Knowledge graph

ABSTRACT

To address the critical challenges of real-time responsiveness and autonomous evolution in highly dynamic flexible production lines, this paper proposes a novel dynamic modeling framework based on a Large Language Model-driven Multi-Agent System (LLM-MAS). Departing from conventional digital twin (DT) paradigms constrained by static configurations, our approach facilitates seamless synchronization and adaptive evolution by decomposing the DT into three interlinked dimensions: knowledge, behavior, and geometry. Leveraging the semantic reasoning of LLMs, an evolving knowledge graph is first synthesized from heterogeneous multi-source data to empower agents with deep situational awareness. Building upon this, a task-driven coordination mechanism dynamically generates and reconstructs behavior models, enabling robust, goal-oriented logic amidst environmental disturbances. This behavioral intelligence further guides the spatio-temporal reconfiguration of geometric models, effectively translating high-level semantic decisions into physical structural and parameter adaptations. Experimental results from a parts defect monitoring and sorting line demonstrate that, compared to conventional methods, the proposed approach reduces changeover and recovery times by 47% and significantly reduces the false positive rate, proving that the LLM-MAS framework provides a robust foundation for the autonomous operation of next-generation smart factories.

1. Introduction

Currently, the global manufacturing industry is undergoing a profound transformation towards intelligence and customization, which demands unprecedented flexibility and response speed from production lines [1,2]. However, traditional rigid production lines, characterized by solidified processes and difficult adjustments, are no longer capable of effectively coping with the high dynamism of market demands. Consequently, the evolution towards flexible production lines that combine high efficiency with high elasticity has become key to the upgrading of the manufacturing sector [3]. In this context, Digital Twin (DT) technology, by constructing a high-fidelity, real-time mapping between the physical world and virtual space, provides a revolutionary technological pathway for the intelligent and flexible upgrading of production lines [4,5]. A Digital Twin is not merely a simple three-dimensional model, it comprehensively describes the multi-dimensional characteristics of its physical counterpart—including geometry, behavior, and rules [6]. It serves as a virtual replica that maintains a bi-directional, real-time data flow with its physical entity, capable of continuously adapting to operational changes and predicting future states [7].

However, when Digital Twin technology is applied to highly dynamic and complex flexible production lines, its dynamic modeling process still faces a series of critical challenges that limit the full realization of its efficacy:

(1) Challenges in Real-time Responsiveness and Fidelity: Flexible production lines are characterized by frequent changes in process paths, dynamic adjustments of equipment states, and random disturbances in material flow. Traditional DT modeling methods exhibit significant latency in real-time capturing of these dynamic changes and accurately updating them to the virtual model. This latency often leads to a mismatch between the physical and virtual worlds (virtual-real distortion), which in turn affects the accuracy of analysis and decision-making [8,9].
(2) Challenges in Adaptability and Evolution: Faced with production line reconfiguration or process innovation driven by flexible demands, traditional DT models are typically constrained by static structures and preset behavioral logic. These models struggle to

* Corresponding author at: Donghua University China.
E-mail address: bao@dhu.edu.cn (J. Bao).

https://doi.org/10.1016/j.dte.2026.100099
Received 16 January 2026; Received in revised form 23 February 2026; Accepted 24 March 2026
Available online 24 March 2026
2950-550X/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

self-adjust and optimize according to the actual evolution of the production line, lacking the necessary adaptive learning and autonomous evolutionary capabilities to maintain their effectiveness continuously [10,11].

(3) Challenges in Semantic Understanding and Intelligent Decision-making: Flexible production lines involve multi-level complex relationships ranging from units and modules to the system level, alongside massive amounts of heterogeneous data. Traditional modeling approaches find it difficult to achieve efficient fusion of cross-scale information and the extraction of deep semantic knowledge. Consequently, the construction of advanced twin applications capable of providing intelligent diagnosis and predictive maintenance is hindered by a lack of effective knowledge reasoning and collaborative mechanisms [12].

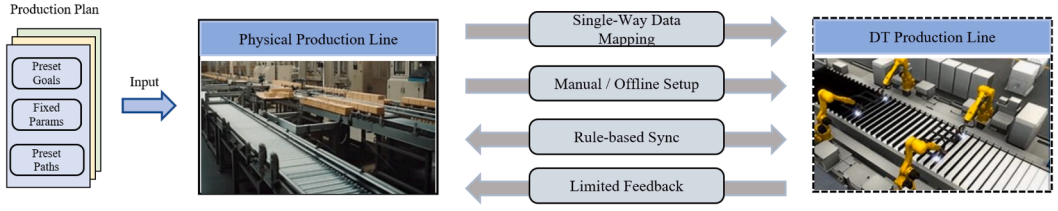
In recent years, the deep integration of LLMs with MAS has introduced a novel technical paradigm for resolving these dilemmas. Traditional rule-based agents fail when encountering unforeseen disturbances, and pure data-driven deep learning models often function as black boxes lacking interpretability. LLM-MAS overcomes these limitations through zero-shot generalization and deep semantic comprehension [13]. They enable agents to interpret unstructured multi-source data, autonomously reason through unprecedented anomalies, and dynamically decompose complex tasks into executable control scripts without prior hardcoding or extensive retraining [14].

Meanwhile, MAS provides the essential framework for distributed coordination and parallel processing. By organically combining the cognitive intelligence of LLMs with the collaborative architecture of MAS, agent groups can dynamically negotiate and execute system-level modeling tasks. Therefore, this approach is considered the optimal path to fundamentally enhance the response speed, adaptive capability, and overall intelligence level of DT models regarding the dynamics of flexible production lines [15].

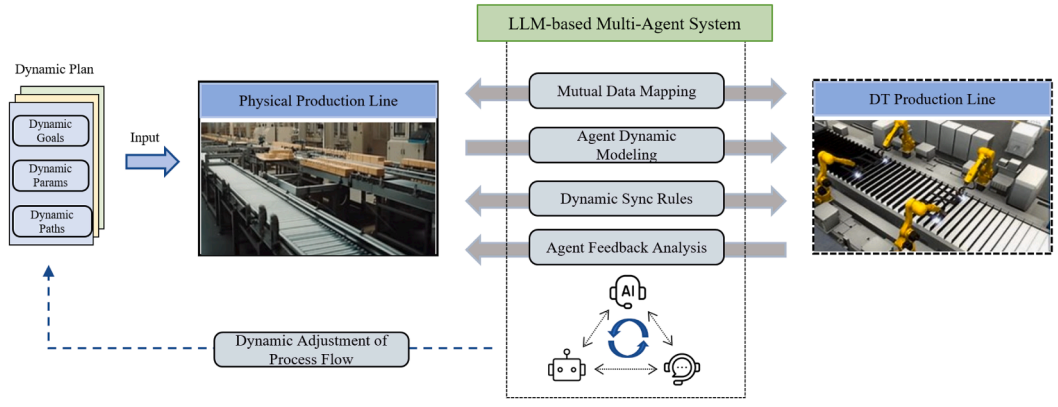
The challenges mentioned above highlight existing technical bottlenecks, while Fig. 1 intuitively illustrates the evolution of the solution strategy. As shown in Fig. 1(a), traditional digital twin modeling paradigms, constrained by solidified data processing links and a reliance on manual intervention, struggle to support the real-time construction and deep interaction of models. In contrast, as shown in Fig. 1(b), the multi-agent-based dynamic modeling approach proposed in this paper utilizes LLM-MAS to break through these limitations.

To systematically implement this research concept, this paper proposes a flexible production line digital twin dynamic modeling method based on LLM-MAS. This method constructs a hierarchical agent collaboration framework and, through a knowledge-behavior-geometry ternary synergistic mechanism, directly maps the agents' autonomous perception and decision-making processes to the construction and evolution processes of the twin model. Specifically, to address the real-time issue, the behavior model relies on hierarchical agents to achieve the step-by-step aggregation and logic synchronization of underlying equipment states, ensuring model fidelity. To address the adaptability issue, the geometric model introduces an autonomous planning mechanism based on a Hierarchical Task Tree (HTT), which drives the adaptive update of the model under production line reconfiguration through structured task flows, replacing traditional manual reconfiguration. To address the semantic understanding issue, the $M_{Knowledge}$ utilizes a Graph Agent to dynamically refine semantic knowledge from multi-source heterogeneous data and construct a dynamic knowledge graph, providing deep contextual support for the system's reasoning and decision-making.

The remainder of this paper is organized as follows: Section 2 reviews the current research status of digital twin production line modeling and large language model multi-agent systems. Section 3 elaborates on the methodology of the paper, including the task-driven hierarchical modeling system, the agent-driven ternary model constitution mechanism, and the agent-oriented asset knowledge graph construction and dynamic evolution method; furthermore, this section focuses on agent-based unit modeling, collaborative decision-making mechanisms, and unit reorganization and multi-scale model update strategies based on the HTT. Section 4 verifies the feasibility of the



(a) Traditional Method



(b) Our Method

Fig. 1. Comparison between the method proposed in this paper and traditional methods.

proposed method through specific case experiments and provides a comparative analysis of its advantages in coping with flexible production challenges. Section 5 summarizes the full text and provides an outlook on future research directions.

2. Literature review

2.1. Digital twin production line modeling

With the in-depth development of Intelligent Manufacturing and Industry 4.0 concepts, digital twin production line modeling has become a core content of research on intelligent manufacturing systems [16]. Building a high-fidelity, highly dynamic response twin model helps realize real-time perception, intelligent decision-making, and adaptive optimization of production systems. Currently, research on digital twin production line modeling mainly focuses on the following aspects: First, the real-time fusion and processing of multi-source heterogeneous data, aiming to ensure that the twin model can accurately perceive the instantaneous state of the physical production line [17]; Second, high-fidelity multi-dimensional modeling technology, including the precise construction and integration of geometric, physical, behavioral, and rule models [18]; Third, model-driven intelligent analysis and optimization [19], such as fault diagnosis, performance prediction, and process optimization based on twin data; Fourth, the dynamic evolution and adaptive mechanism of the twin model, exploring how to enable the model to autonomously update and evolve with changes in the physical production line.

Unlike traditional digital simulation models, production line digital twin modeling emphasizes using a high-fidelity virtual workshop model as a real-time mirror of the physical workshop, utilizing real-time production data to drive the synchronous fusion of the virtual model and physical entities [20]. Through this process, information such as simulation, production flow, status data, and geometric forms can accurately reflect the real-time production situation of the physical workshop, thereby achieving dynamic synchronization and closed-loop interaction between the physical and virtual workshops.

Digital twin production line modeling aims to perform a comprehensive and fine-grained digital expression and integration of equipment, processes, material flows, and real-time data in the production line. It precisely reproduces the operating status of the physical production line through a virtual model and empowers real-time monitoring and intelligent optimization. For instance, Liu et al. [21] realized centralized management of multi-source data and optimization of machining sequences by establishing a feature-process-machine tool hyper-network model in a digital twin workshop. Yi et al. [22] proposed a digital twin reference model for intelligent assembly process design, utilizing 3D models of equipment to optimize production line layout to shorten the travel distance between processes.

Although existing research has improved model accuracy and application scope to a certain extent, system-level dynamic modeling methods remain insufficient regarding the unique high dynamics and complexity of flexible production lines [23]. For example, regarding the expression of hierarchical relationships of objects and cross-scale information interaction, Xie et al. [24] proposed a digital twin geometric-physical-behavioral-information control four-dimensional fusion modeling method and built an operation and maintenance management system. Liu et al. [25] also proposed a multi-level fusion framework to support the fusion of real-time data and twin models from the unit level to the system level. However, how to achieve dynamic, intelligent collaboration between these hierarchical models and how to improve data fusion efficiency to support rapid adaptive evolution of the model remain key challenges facing current research and are the focus of future studies.

Therefore, despite significant progress in digital twin technology in the field of production line modeling, there is still considerable room for improvement in traditional modeling methods when coping with the dynamic demands of flexible production. Specifically, improvements are needed in the efficiency of real-time model construction and updating, the adaptive capability to production line changes, and the level of intrinsic intelligent decision-making and autonomous evolution of the model. There is an urgent need to explore a more agile and intelligent new paradigm for dynamic modeling to meet the pressing needs of flexible production for rapid response, precise mapping, and intelligent optimization of digital twin models.

2.2. Large language model-based multi-agent systems

To address the challenges in digital twin dynamic modeling mentioned above, especially to improve the adaptability and intelligence level of models, the LLM-MAS has shown enormous application potential. Traditional MAS usually rely on preset rules for collaboration, with relatively solidified behavioral patterns and limited learning and adaptive capabilities [26]. In recent years, generative artificial intelligence based on the Transformer architecture has made breakthrough progress [27], giving birth to powerful LLMs such as GPT-5.2,Claude Opus 4.5 and Gemini 3 Pro [28,29]. These LLMs demonstrate unprecedented capabilities in natural language understanding, knowledge reasoning, code generation, and complex logical dialogue. Their core self-attention mechanism enables LLMs to efficiently process high-dimensional information and capture long-distance dependencies between contexts, laying a solid technical foundation for building more advanced and intelligent MAS.

LLM-MAS is a distributed system composed of multiple agents possessing an LLM cognitive kernel. The introduction of LLMs enables agents to more effectively understand complex environmental information (including unstructured data), share knowledge, perform deep reasoning, and make collaborative decisions, opening up a new paradigm for the construction and application of MAS [30]. A single agent in LLM-MAS not only possesses a high degree of autonomy, capable of decomposing complex tasks into sub-goals and planning execution paths, but also can learn from interaction experiences and perform self-reflection [31,32], thereby continuously optimizing decision quality. Furthermore, addressing the limitation of knowledge timeliness inherent in large language models (i.e., the inability to access new knowledge after the model training cutoff date), agents in LLM-MAS can acquire and integrate the latest information by flexibly calling external tools, accessing real-time databases and knowledge bases. This allows them to maintain context consistency and behavioral effectiveness in dynamically changing environments [33]. Overall, LLM-MAS, by emphasizing the diversity of agent roles, the intelligence of interactions, and the collectivity of decision-making, enables the handling of complex dynamic tasks that were previously difficult to manage through the collaboration of autonomous agents [34].

Recently, preliminary explorations of LLM and agent technologies in industrial manufacturing have emerged. Foundational works have begun to bridge LLMs and Industry 4.0; for instance, Lee and Su [35] proposed a unified industrial large knowledge model framework designed to integrate complex manufacturing data, models, and domain expertise. Building upon this, the paradigm of "Agentic AI" has been introduced to extend traditional task-specific AI through LLM-driven advanced reasoning, dynamic planning, and multi-agent collaboration in smart manufacturing [36]. While these pioneering studies provide vital strategic guidance and demonstrate feasibility through applications like conversational fault diagnosis, current practical implementations often still treat LLMs primarily as auxiliary text-processing tools or isolated decision nodes for translating natural language into robotic code [37,38]. In principle, these existing agent-based or knowledge-graph (KG)-driven DT approaches rely on static KGs as passive query databases and utilize pre-programmed negotiation protocols for isolated task execution.

The innovation of this study lies in positioning the LLM-MAS not merely as an external reasoning tool, but as the continuous operating

engine that seamlessly binds the Knowledge-Behavior-Geometry (K-B-G) dimensions of the digital twin. Our framework fundamentally shifts the traditional paradigm: the Graph Agent autonomously extracts semantic knowledge from real-time data streams to self-evolve the knowledge graph, eliminating the need for manual ontology updates. This dynamic semantic understanding is instantly translated into the Behavior model via adaptive task decomposition, which directly drives the spatio-temporal reconfiguration of the Geometry model through the Hierarchical Task Tree. Through this K-B-G ternary synergistic mechanism, the agents' continuous perception and collaborative negotiation directly manifest as real-time structural reconstruction and parametric evolution of the digital twin.

In the field of dynamic modeling for flexible production line digital twins, these characteristics of LLM-MAS foreshadow new solution paths. For example, agents can utilize the understanding capability of LLMs to parse dynamic production instructions and multi-source heterogeneous production line data. Through negotiation between agents, they can dynamically construct and update the structure and parameters of the digital twin model, and even autonomously learn and optimize the model's behavioral logic. The natural language interaction, dynamic reasoning, and continuous learning capabilities possessed by LLM-based agents are expected to significantly improve the digital twin model's response speed to production line changes, its flexibility in adapting to flexible demands, and the intelligence level of auxiliary production decision-making [39]. Based on this background, this paper is dedicated to exploring how to deeply utilize Large Language Models to enhance the capabilities of multi-agents in perception, cognition, decision-making, and collaboration, and apply this to the dynamic construction and adaptive evolution of digital twin models for flexible production lines, with the aim of breaking through the limitations of traditional modeling methods.

3. Methodology

3.1. Hierarchical framework overview

3.1.1. Task-driven collaborative mechanism

To address the complexity and real-time requirements inherent in the dynamic modeling of digital twins for flexible production lines, this study proposes and constructs a task-driven LLM-MAS dynamic modeling framework, as shown in Fig. 2. The core philosophy of this framework is to realize the dynamic, precise modeling and real-time responsiveness of flexible production line digital twins through hierarchical collaboration and autonomous decision-making among agents.

First, dynamic processing tasks generated by the flexible production line (e.g., processing a specific batch of workpieces, T) are input to the Line Agent. As the highest level of the system, the Line Agent is responsible for the overall planning and preliminary decomposition of the received dynamic tasks, transforming them into a series of ordered modular sub-tasks (e.g., T_1, T_2, …, T_n). Based on the decomposition results, the Line Agent assigns the corresponding modular sub-tasks (such as T_1) to the corresponding Module Agents (such as Module Agent 1). Each Module Agent corresponds to a specific physical module in the production line (e.g., machining module, assembly module, quality inspection module) and is responsible for managing and coordinating the production activities of that module. Upon receiving the sub-task (T_1) assigned by the upper layer, the Module Agent (e.g., Module Agent 1) further decomposes the task into finer-grained unit tasks (e.g., $T_{1.1}$, $T_{1.2}$, …, $T_{1.n}$) based on the specific functions and current status of the module it oversees, and then assigned to the corresponding Unit Agents (e.g., UA_1, UA_2, etc.). The Unit Agents interact directly with the physical execution units on the production line and their corresponding virtual entities in the digital twin model, completing specific operations such as machining, transportation, and inspection by controlling these physical units, while synchronously updating the state of the twin model.

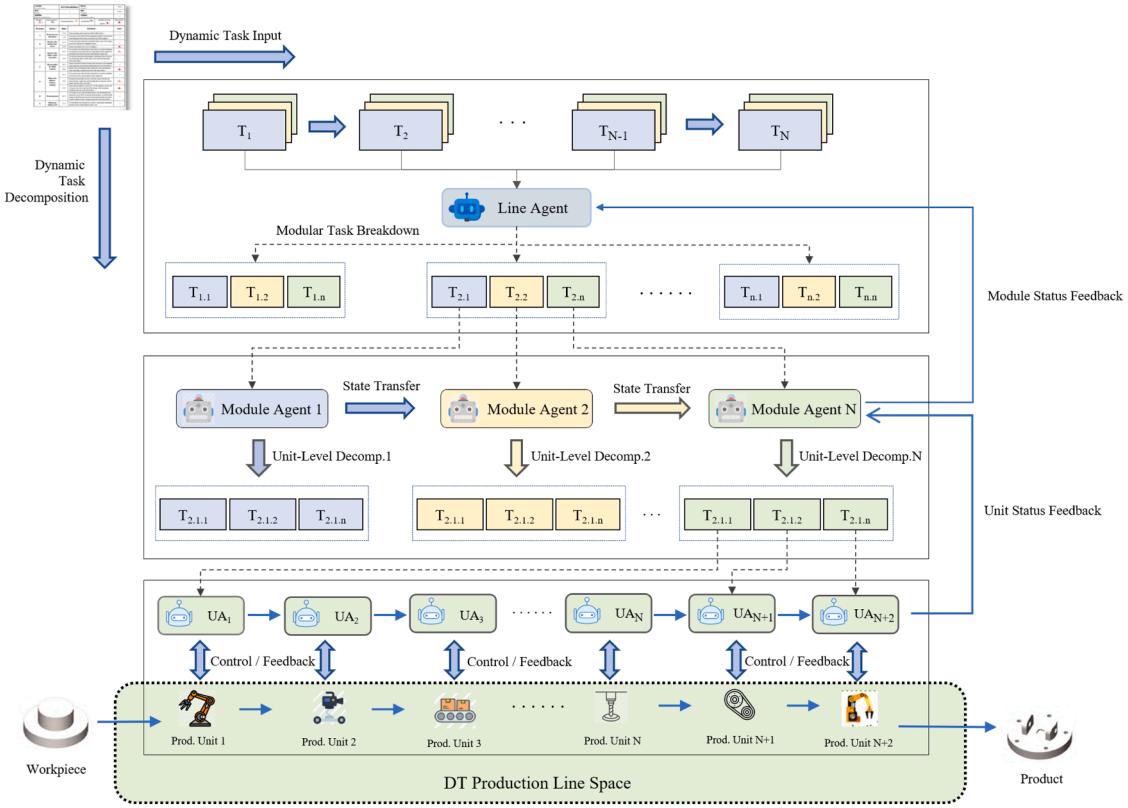


Fig. 2. Task-driven LLM-MAS dynamic modeling framework for production lines.

In the task execution phase, a bottom-up multi-level state feedback mechanism is constructed. Unit Agents upload execution data in real-time to the module level for state aggregation and verification; based on the serial process logic of the production line, Module Agents cooperatively trigger downstream modules and transmit status summaries upwards after confirming the closure of local tasks. As the terminal modules complete their feedback, the Line Agent determines that the current task has achieved a global closed loop and initializes the next round of task planning, thereby forming a dynamic modeling cycle of execution-feedback-iteration.

Through the aforementioned hierarchical mechanism of task decomposition, allocation, execution, and feedback, agile response and efficient execution of dynamic tasks in flexible production lines are achieved. More importantly, this process itself constitutes the dynamic construction mechanism of the digital twin model: every task execution on the physical production line drives the state updates, behavioral simulation, and relational evolution of the corresponding virtual entities in the digital twin model through multi-agent collaboration, thereby ensuring that the digital twin model can accurately and continually reflect the actual status and processing progress of the physical production line in real time.

3.1.2. LLM-driven agent architecture

In the framework of this study, agents based on LLMs serve as the core driving units for executing dynamic modeling tasks. Each agent is essentially an autonomous computational entity driven by an LLM, integrating four core capability modules: Perception, Memory, Planning, and Action, as shown in Fig. 3.

The execution logic of the agent adheres to a continuous, cyclic decision-making process. First, information is received from the external environment via the perception module. The agent then utilizes its LLMs core to process this information for planning—specifically, by comprehending the current context, conducting logical reasoning, and decomposing complex goals into a sequence of executable steps. Within this architecture, the LLM serves not merely as an alternative computational tool, but as an indispensable kernel for handling the inherent uncertainties of flexible manufacturing. Traditional rule-based algorithms or supervised learning models often struggle with the unstructured nature of such environments. By contrast, the LLM provides critical advantages across four operational dimensions. For knowledge extraction, it enables the zero-shot parsing of heterogeneous, unstructured data (e.g., equipment manuals and real-time error logs) into structured ontology triples, bypassing the need for extensive manual annotation. During reasoning, its deep semantic comprehension allows the agent to contextualize unforeseen production disturbances that would typically break rigid algorithmic logic. For task decomposition, the LLM autonomously translates high-level, ambiguous production orders into precise, executable sub-tasks without relying on pre-defined, exhaustive task trees. Finally, in coordination, it replaces hard-coded communication protocols with flexible, context-aware semantic negotiation, dynamically resolving resource conflicts between agents in real-time.

Based on these planning outcomes, the agent makes decisions and invokes external tools to initiate the action phase. The resulting outcomes and acquired experiences are subsequently stored in the memory module. Through such dynamic interaction with the environment and continuous learning, the agent achieves continuous iteration and enhancement of its capabilities. The evolution of the agent is defined as follows:

$$Agent_{i+1} = Evolve(Agent_i) = f(Process_i, Action_i, Memory_{i+1}) \quad (1)$$

This evolution function f is driven by three core components: Core Processing ($Process_i$), which involves using the LLM combined with current input and historical memory for reasoning and decision-making; Autonomous Action ($Action_i$), which involves interacting with the environment by calling external tools at appropriate times; and Memory Update ($Memory_{i+1}$), which involves updating the memory bank with the experiences and conclusions from the current interaction. The state of the agent at time $i+1$ is the result of the evolution of its state $Agent_i$ at time i.

A high-fidelity digital twin must be capable of simultaneously capturing its static physical and logical structure, dynamic operational behavior, and precise geometric form. Therefore, this study deconstructs the digital twin model of the flexible production line into a dynamically evolving ternary tuple:

$$DT_{prodline} = \{M_{knowledge}, M_{behavior}, M_{geometry}\} \quad (2)$$

Where:

- Knowledge Model ($M_{Knowledge}$): Serving as the semantic backbone of the twin, it is constructed with a dynamic knowledge graph at its core. It is responsible for abstracting elements of the physical production line—such as equipment, materials, and processes—and their interrelationships into structured semantic information.
- Behavior Model ($M_{behavior}$): Serving as the operational logic of the twin, it is driven by the collaborative decision-making flow of the multi-agent system. By simulating top-down task decomposition and instruction transmission, as well as bottom-up state feedback and dynamic negotiation among agents, it replicates the real-time operational mechanism of the production line.
- Geometric Model ($M_{geometry}$): Serving as the visual carrier of the twin, it is the external manifestation of the $M_{knowledge}$ and $M_{behavior}$. It externalizes the internal mechanisms of the knowledge and behavior models into a perceptible form, achieving a high-fidelity mapping of the real production line's morphology through 3D modeling of its physical structure, spatial topology, and assembly relationships.

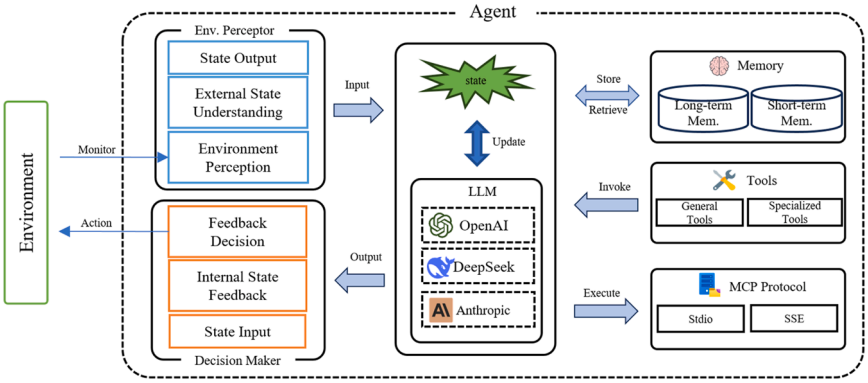


Fig. 3. Agent perception and decision-making process.

3.2. Dynamic knowledge graph modeling

The $M_{Knowledge}$ serves as the semantic foundation for environmental perception, task understanding, and collaborative decision-making within the LLM-MAS. To address the static and lagging issues inherent in traditional knowledge graph construction methods when dealing with the dynamics of flexible production lines, a dynamic construction and interaction method for production line knowledge graphs based on LLMs is proposed. By introducing a dedicated Graph Agent, the continuous evolution and full lifecycle support of $M_{Knowledge}$ within the dynamic production line environment are achieved.

As illustrated in Fig. 4, the core workflow of the Graph Agent consists of two primary phases: First, guided by predefined ontology models, it dynamically constructs and evolves the knowledge graph from heterogeneous production line data sources. Second, it provides intelligent knowledge query and decision support services to other hierarchical agents (i.e., Line, Module, and Unit agents) within the system. The inputs and outputs of the Graph Agent are strictly determined by predefined fine-tuned prompts and structured output formats, respectively, to ensure the consistency and accuracy of message exchange between agents, the specific definitions of these input prompts and structured outputs are detailed below.

3.2.1. Ontology-guided graph generation

3.2.1.1. Hierarchical ontology definition. Regarding dynamic knowledge extraction and evolution, to ensure the systematicity and consistency of knowledge representation, this method first constructs a three-layer knowledge graph ontology corresponding to the physical production line hierarchy: The Production Line Layer, the Module Layer, and the Unit Layer. The Production Line Layer describes macroscopic production processes and plans; the Module Layer defines the composition and processes of various functional modules (such as assembly and inspection); the Unit Layer refines details to the specific parameters and status of each device and sensor. This hierarchical structure makes $M_{Knowledge}$ clear and organized and also provides explicit schema guidance for subsequent automated knowledge extraction:

$$S_{line} = LS \cup MS \cup US \tag{3}$$

Specifically, the entity and relationship elements shown in Table 1 are extracted, and the Line Schema (*LS*), Module Schema (*MS*), and Unit Schema (*US*) are defined respectively.

The *LS* is defined by Formula 4, containing line layer elements E_{line}, line layer features F_{line}, and line layer relationships R_{line}, expressed as follows:

$$LS = \{E_{line} \cup F_{line} \cup R_{line}\} \tag{4}$$

$$E_{line} = \{PL \cup PP \cup GS \cup ProT\} \tag{5}$$

$$F_{line} = \{PT \cup PLL \cup PO \cup POS \cup PTS \cup ES\} \tag{6}$$

$$R_{line} = \{CM \cup GPL \cup RPL \cup PPT\} \tag{7}$$

The *MS* is defined by Formula 8, containing module layer elements E_{module}, module layer features F_{module}, and module layer relationships R_{module}, expressed as follows:

$$MS = \{E_{module} \cup F_{module} \cup R_{module}\} \tag{8}$$

$$E_{module} = \{Module \cup Process \cup Material\} \tag{9}$$

$$F_{module} = \{EG \cup MT \cup ML \cup MP \cup MD \cup PN \cup PD \cup QS\} \tag{10}$$

$$R_{module} = \{CU \cup BTPL \cup PP \cup RM \cup SM \cup UEG\} \tag{11}$$

The *US* is defined by Formula 12, containing unit layer elements E_{unit}, unit layer features F_{unit}, and unit layer relationships R_{unit}, expressed as follows:

$$US = \{E_{unit} \cup F_{unit} \cup R_{unit}\} \tag{12}$$

$$E_{unit} = \{Unit \cup Equipment \cup Sensor \cup Status\} \tag{13}$$

$$F_{unit} = \{UT \cup EM \cup ST \cup UP \cup OP\} \tag{14}$$

$$R_{unit} = \{BTM \cup EP \cup ME \cup RS \cup HCP\} \tag{15}$$

The Three-Layer Production Line Hierarchical Knowledge Graph (Three-Layer PL-HKG) systematically organizes production line knowledge using the *LS-MS-US* three-layer ontology structure shown in Fig. 5. Knowledge at different levels serves as the external perception source for the corresponding level agents in the subsequent $M_{behavior}$.

The *LS* describes macroscopic global information and production plans, providing global situational awareness and macro-decision support for Line Agents. The *MS* describes the composition, processes, and

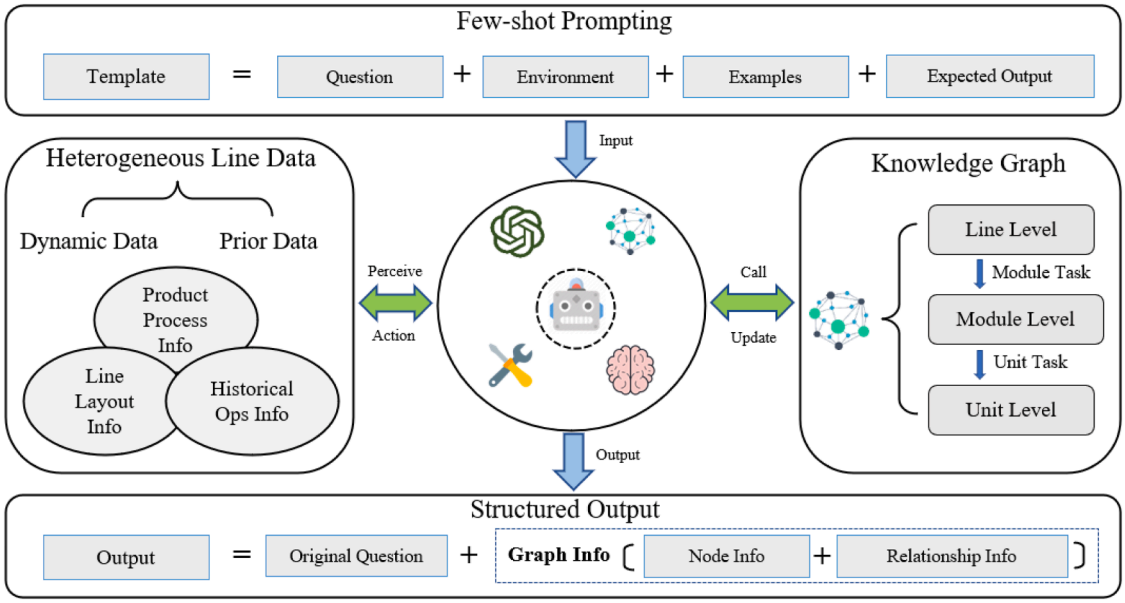


Fig. 4. Dynamic knowledge graph construction workflow.

Table 1
Hierarchical knowledge graph schema definition for production line.

Hierarchy	Entity & Attribute	Description	Relationship	Description
Line Schema	PL	Production Line	CM	ContainsModule
	PP	Production Plan	GPL	GuidesProductionLine
	GS	Global operating status	RPL	ReflectsProductionLine
	ProT	Product type produced	PPT	ProducesProductType
	PT	Production line type		
	PLL	Production line layout		
	PO	Planned output		
	POS	Production line operating status		
	PTS	Production task status		
	ES	Environmental status		
Module Schema	Module	Module	CU	ContainsUnit
Unit Schema	Process	Process	BTPL	BelongsToProductionLine
Other	Material	Material	PP	PrecedesProcess
	EG	Equipment Group	RM	RequiresMaterial
	MT	Module Type	SM	SuppliesMaterial
	ML	Module Location	UEG	UtilizesEquipmentGroup
	MP	Module Performance Parameters	BTM	BelongsToModule
	MD	Material Demand	EP	ExecutesProcess
	PN	Process Name	ME	MonitorsEquipment
	PD	Process Description	RS	ReflectsStatus
	QS	Quality Standard	HCP	HasConfigurationParameter
	Unit	Unit		
	Equipment	Equipment		
	Sensor	Sensor		
	Status	Status		
	UT	Unit Type		
	EM	Equipment Model		
	ST	Sensor Type		
	UP	Unit Pose		
	OP	Operating Parameters		
	Id	Unique Identifier		
	Name	Name		
	Weight	Weight		
	Volume	Volume		

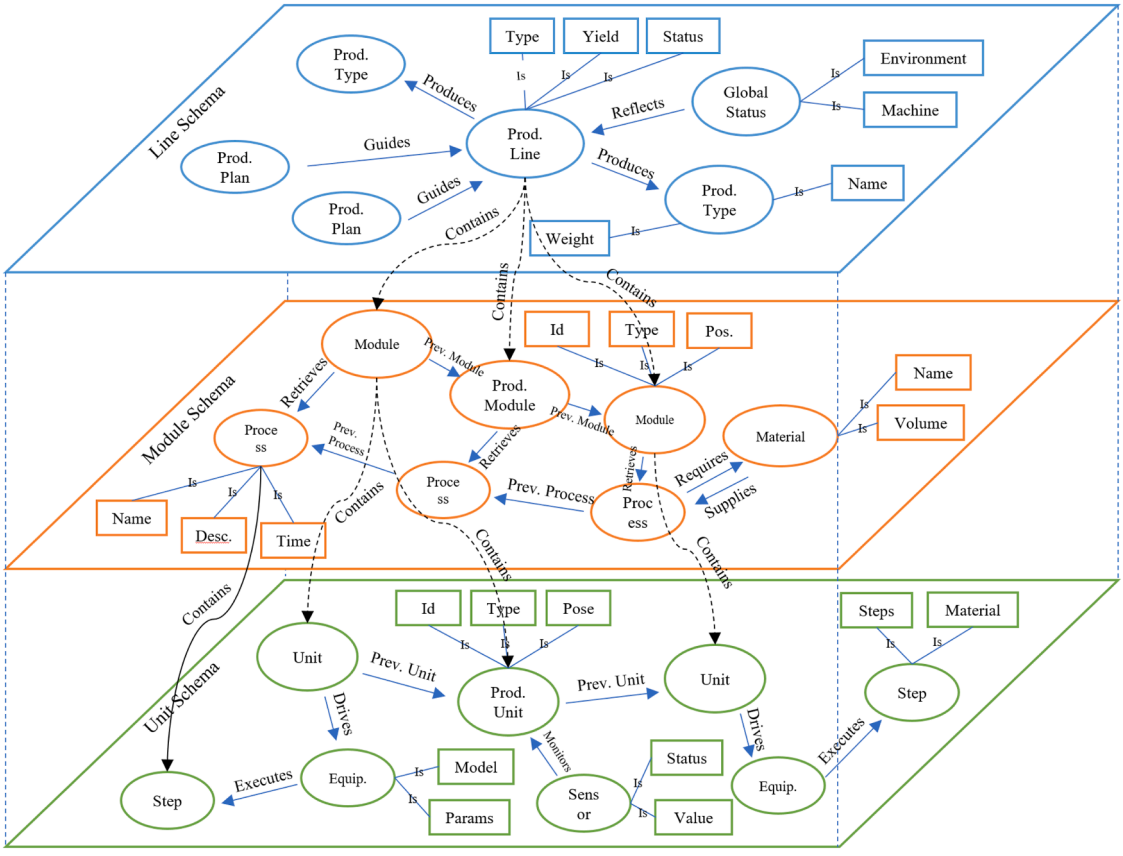


Fig. 5. LS-MS-US three-layer production line knowledge graph.

resource configuration of functional modules, providing intra-module resource scheduling and process coordination knowledge for Module Agents. The *US* describes detailed information and status data of equipment units, providing the basis for precise control and status monitoring at the equipment level for Unit Agents.

3.2.1.2. Real-time knowledge updating. Under the guidance of this ontology, the Graph Agent is responsible for refining semantic knowledge from various heterogeneous data sources of the production line. As shown in Fig. 6, knowledge graphs constructed by traditional knowledge extraction methods (such as manual construction or deep learning-based sequence labeling models) are often static. Once built, they are difficult to adapt to the dynamic changes of flexible production lines, leading to high maintenance costs and timeliness issues. To solve this problem, this method utilizes the LLM kernel of the Graph Agent to continuously parse static data (e.g., CAD models, equipment manuals), semi-dynamic data (e.g., process flow documents, BOM lists), and dynamic data (e.g., sensor readings, MES work orders) of the production line. The agent can automatically extract key entities, attributes, and their relationships that conform to the predefined ontology from unstructured or semi-structured information and transform them into structured "Head Entity-Relation-Tail Entity" triples.

The lifecycle of $M_{Knowledge}$ includes two stages: Baseline Construction and Dynamic Evolution. First, by processing existing static and semi-dynamic data, an initial version of the knowledge graph, namely the baseline graph KG_0, is formed. This process can be formally described as:

$$KnowledgeGraph_0 = \sum_{i=1}^{n} LLM(Segment_i(PreparedData)) \tag{16}$$

Where *PreparedData* is the pre-processed stock data, $Segment_i$ is the data splitting function, and the LLM is responsible for extracting triples from each data shard. After the construction of the baseline graph KG_0, it becomes the starting point for the dynamic evolution process. The Graph Agent continuously monitors the dynamic data flow of the production line. When the production line status changes, it automatically processes new or changed data and fuses the new knowledge into the existing graph through the following dynamic evolution paradigm:

$$KG_{i+1} = Update(KG_i, LLM_Extraction(PreparedData_{i+1})) \tag{17}$$

3.2.2. Semantic knowledge retrieval

Agents at various levels in the $M_{behavior}$ need to obtain specific production line information from the $M_{Knowledge}$ to continuously update their own memory and achieve dynamic synchronization with the physical production line.

To ensure that the output of the Graph Agent conforms to the expected format and semantic information, and can correctly output the relationships between production line nodes, ensuring the result is concise enough to serve directly as input for another agent, it is necessary not only to standardize the output results but also to improve the input Prompt to guide the Graph Agent to output content in a specific format.

Using Few-shot Prompting can significantly improve the accuracy of the large model's output format. The input template for the Graph Agent's Prompt is as follows:

```
Input:Prompt for Graph Agent
Hierarchy_Level: {Asset's Hierarchy Level}
  AssetName: {Asset's name}
  Coordinate: {Asset's Spatial Coordinate}
  Relationship:{relationship define}
  Instance: {Example}
  Question: Please search the current location and relevant status information of the
  corresponding production line asset in the {Hierarchy_Level} based on {AssetName}
  and {Coordinate}, as well as the relevant status of surrounding production line
  assets. Sort and filter by relevance according to the {Relationship} with
  {AssetName}. The filtered production line assets must be contained within
  {Instance}.
  DesiredOutput:
[TargetAsset({name},{newCoordinate},{position},{status})-Relationship-
  TargetAsset({name}, {newCoordinate}, {position}, {status}), ……]
```

Where *Hierarchy_Level* indicates the hierarchy type, divided into the Line Layer describing topological relationships, the Module Layer specifying the range of functional modules, and the Unit Layer limiting the coordinate range at the equipment level. *AssetName* defines the asset name, using the line identifier, module number, or unique equipment ID depending on the level. *Coordinate* specifies spatial coordinates, including relative layout coordinates for the line layer, absolute coordinates for the module layer, and 3D spatial poses for the unit layer. *Relationship* defines the relationship type, including production cycle dependencies for the line layer, material supply relationships for the module layer, and fault propagation relationships for the unit layer. *Instance* provides instance constraints to limit the possible range of relationships. *Question* defines the dynamic query logic, returning corresponding association analysis results based on different hierarchical requirements while *DesiredOutput* specifies the standardized output format .

Setting the prompt alone cannot fully constrain the agent's output format. It is still necessary to strictly regulate the output format. Through the *Structured Output* of the LLMs, the output format of the Graph Agent can be strictly specified to remain consistent with the output format defined by the Few-shot Prompt. The defined output format for the Graph Agent is as follows:

```
Output:Graph Agent Output
{
    "question": "User's question",
    "answer": [
    {
      "node_name": "Node Name",
      "node_coordinate": "Production Line Asset Coordinate",
      "node_status": "Production Line Asset Status",
      "node_relationship": "Relationship with the queried node",
      "node_description": "Node Description"
    },
    …
    ]
}
```

After the Graph Agent calls the graph database query tool to obtain the corresponding list of triple information, it needs to filter out the production line assets required to complete the current process based on

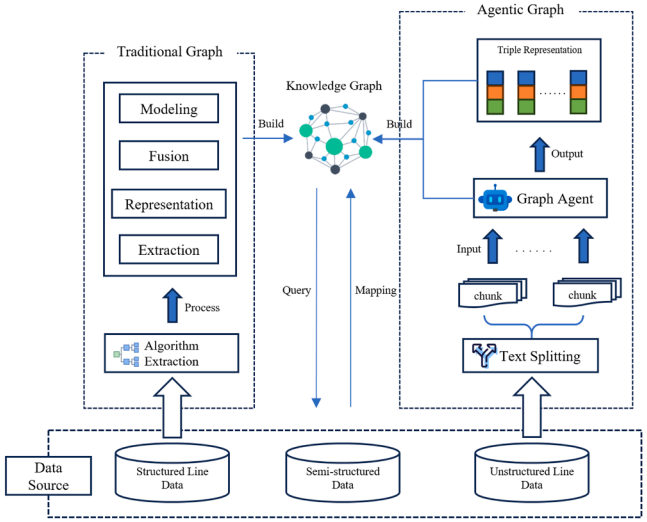


Fig. 6. Dynamic graph knowledge extraction.

the current work order information. The process is as follows: First, the work order information and the queried triple node information are converted into vector representations through an embedding model:

$$VS <> VectorizedSentence = Embedding(Sentence) \tag{18}$$

Then, the Cosine Similarity (Formula 19) is used to calculate the semantic similarity between two text vectors x and y:

$$Similarity(x,y) = \cos\langle x,y\rangle = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}} \tag{19}$$

Based on this, the similarity values are sorted. When the similarity metric exceeds a preset threshold, the queried production line asset is considered a necessary device for completing the current process:

$$IsRelevant(t, WorkOrderInfo) = Similarity(VS, Vectorize(t)) > Threshold \tag{20}$$

Finally, all triples with strong relevance are collected to form the final filtered result list, which is output to the agent in the $M_{behavior}$ that requested perception of the production line status.

The overall workflow Algorithm 3 for the Graph Agent, from receiving a graph query request to outputting the final result, is shown below:

```
Algorithm 3: Graph Agent Workflow
Input: GraphQueryRequest_i
Output: FilteredTriplesList
  // Initialize Agent State
  GraphAgent = Initialize_Agent(GraphQueryRequest_i.context)
  // Knowledge Understanding and Transformation Layer
  CypherQuery = LLM_Processor(
    prompt_template = Hierarchical Prompt Template, // Structured Prompt
containing Few-shot examples
    user_query = GraphQueryRequest_i,
    memory = GraphAgent.Memory // Dynamic memory context
  )
  // Knowledge Retrieval and Processing Layer
  if CypherQuery.valid:
    RawTriples = GraphDB_Tool.execute(CypherQuery) // Execute Graph Database
Query
    // Semantic Relevance Filtering
    ContextVector = Embedding_Tool(WorkOrderInfo.current) // Vectorize Work
Order Info
    FilteredResults = []
    for triple in RawTriples:
      TripleVector = Embedding_Tool(triple.description)
      if cosine_similarity(TripleVector, ContextVector) > θ: // θ: the similarity
threshold
        FilteredResults.append(triple)
    // Structured Result Generation
    StructuredResults = LLM_Processor(
      format_rules = Standardized Output Template, // Enforce JSON format output
      filtered_data = FilteredResults
    )
  // Memory Synchronization and Update
  GraphAgent.update_memory(
    new_context = StructuredResults,
    hierarchy = GraphQueryRequest_i.Hierarchy_Level // Update memory by
production line level
  )
  return StructuredResults // Return format: [Node@Coordinate - Relationship ->
  Associated Node]
```

The algorithm receives graph query requests from other agents and subsequently enters a processing stage centered on LLM. In this stage, the LLM undertakes a dual task: understanding the user's natural language query and converting it into an executable Cypher query statement. The agent executes specific tasks through a predefined set of tools (including graph database query tools, vectorization tools, and filtering tools). After obtaining the raw query results, the algorithm performs semantic similarity filtering based on the current process context information. This process involves vectorizing the work order information and query results, and screening relevant information based on a similarity threshold. Finally, the algorithm formats the filtered results to generate a structured query result and updates the agent's memory module before returning the result, thereby realizing the dynamic update of the $M_{Knowledge}$.

3.3. Collaborative behavioral decision framework

On the semantic foundation provided by the $M_{Knowledge}$, the $M_{behavior}$ is responsible for simulating and driving the dynamic operational logic of the production line. $M_{behavior}$ is not a predefined rigid process but is dynamically constructed through the real-time collaborative decision-making of the hierarchical LLM-MAS. The core of this model lies in the hierarchical design of agents and the task-driven interactive decision-making process.

3.3.1. Multi-level agent structure

To accurately map the physical and organizational structure of the flexible production line, this framework designs the agent system into three functionally distinct levels, as shown in Fig. 7. Agents at each level are assigned different responsibilities and decision-making scopes, and they obtain the information required for decision-making by calling the $M_{Knowledge}$ described in Section 3.2. The decision-making structure, from top to bottom, can be divided into: the Line Agent Layer, the Module Agent Layer, and the Unit Agent Layer.

The Line Agent, as the highest decision-maker of the system, is responsible for global coordination and macroscopic planning. It receives external dynamic production tasks (such as new customer orders), combines them with the understanding of the entire production line in the $M_{Knowledge}$ (such as production capacity, process routes), and decomposes macroscopic tasks into a series of ordered sub-tasks to be completed by different functional modules. It also updates the production line status information obtained from $M_{Knowledge}$ into its own memory. The content of the acquired memory is as follows:

```
ProductionLineAgentState:Internal Memory of Line Agent
{
    "current_production_plan": ProductionPlan, // Current production plan
    "global_status_summary": GlobalStatusSummary, // Global status summary
    "module_performance_metrics": ModulePerformanceMetrics, // Module level
performance metrics
    "pending_tasks": List[Task], // List of pending tasks
    "historical_decisions": List[DecisionRecord], // Historical decision records
    "memory": Memory // Memory module
}
```

Each Module Agent corresponds to a specific physical functional module (such as a loading module, machining module, or inspection module). It receives instructions from the Line Agent, is responsible for further refining module-level tasks into specific operation processes, and

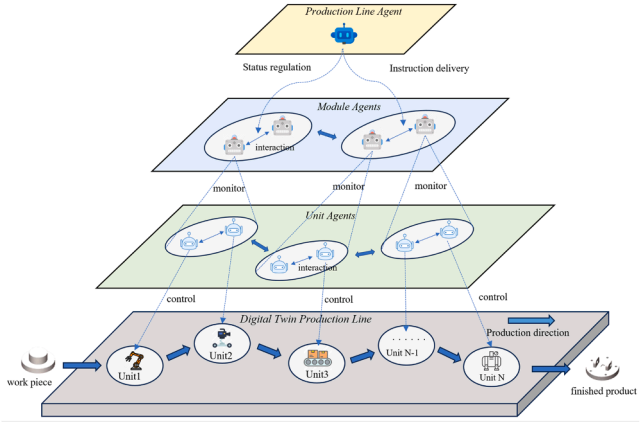


Fig. 7. Agent cooperative decision-making structure.

schedules and optimizes resources within the module (such as equipment and materials). When an abnormality within the module is detected (such as equipment failure or material shortage), the Module Agent performs preliminary local handling, such as starting backup equipment or adjusting process paths, and reports abnormalities that cannot be handled to the superior agent. The corresponding memory content update is as follows:

```
ModuleAgentState:Internal Memory of Module Agent
{
    "module_id": ModuleID, // Module ID (Unique identifier)
    "current_work_order": WorkOrder, // Current work order
    "unit_status_report": UnitStatusReport, // Unit level status report
    "resource_allocation": ResourceAllocation, // Resource allocation status
    "pending_unit_tasks": List[Task], // List of pending unit tasks
    "module_performance_data": ModulePerformanceData, // Module performance
data
    "memory": Memory // Memory module
}
```

As the smallest execution unit of twin modeling, the construction of the Unit Agent is shown in Fig. 8. The Unit Agent is directly bound to one or a group of physical devices (such as robotic arms, conveyor belts, sensors). It receives process instructions from the Module Agent, converts them into bottom-level control commands executable by the equipment, and monitors and feeds back the equipment's running status in real-time.

The Unit Agent receives and precisely executes process tasks issued by the Module Agent, such as controlling a robotic arm to complete grasping and assembly actions, controlling an AGV for material handling, or controlling sensors to collect data. Simultaneously, the Unit Agent directly controls the operation of unit equipment, communicating directly with the physical entity to adjust the motion trajectory of the robotic arm, control the travel speed of the AGV, configure the sampling frequency of the sensors, etc. In addition, the Unit Agent real-time collects operation status data of the unit equipment (such as position, speed, temperature, sensor readings, etc.) and feeds the data back to the Module Agent for status monitoring and decision-making. Its internal memory content is as follows:

```
UnitAgentState:Internal Memory of Unit Agent
{
    "unit_id": UnitID, // Unit ID (Unique identifier)
    "current_task": Task, // Current task
    "unit_status": UnitStatus, // Unit operating status
    "sensor_data": SensorData, // Sensor data
```

(*continued on next column*)

(*continued*)

```
    "equipment_control_parameters": EquipmentControlParameters, // Equipment
control parameters
    "memory": Memory // Memory module
}
```

3.3.2. Closed-loop decision process

The dynamic construction of $M_{behavior}$ is embodied in the task-driven interactive decision-making process, as shown in Fig. 9. This process is essentially a closed-loop control system combining Top-Down Task Decomposition and Bottom-Up Status Feedback.

To ensure efficient and standardized information exchange within this closed-loop system, inter-agent communication is implemented using an asynchronous message-passing mechanism based on the JSON (JavaScript Object Notation) data format. All task assignments, status feedbacks, and negotiation queries between the Line, Module, and Unit agents are encapsulated into a unified JSON structure. The standard communication data format is defined as follows:

```
Communication Protocol for Agents
{
    "message_id": "Unique identifier for the message (e.g., msg_8f3a9b)",
    "sender_id": "ID of the sending agent (e.g., LineAgent_01)",
    "receiver_id": "ID of the receiving agent (e.g., ModuleAgent_02)",
    "message_type": "Type of interaction", // TASK_ASSIGNMENT, STATUS_REPORT,
KNOWLEDGE_QUERY
    "timestamp": "Standardized generation time",
    "priority": "Execution priority level (e.g., LOW, NORMAL, HIGH, URGENT)",
    "content": {
      "task_id": "Associated task identifier",
      "instruction": "Specific semantic instruction or system command",
      "parameters": {
      "target_pose": [150.0, 200.0, 300.0],
      "part_type": "hexagonal_nut"
    },
    "status_data": "Feedback data if applicable (e.g., current completion rate or sensor
values)"
    }
}
```

The communication between the language models and the multi-agent system relies on a structured asynchronous message-passing protocol that bridges cognitive reasoning with physical execution. The language model embedded within each agent processes unstructured environmental data and formulates high-level decisions, which are then

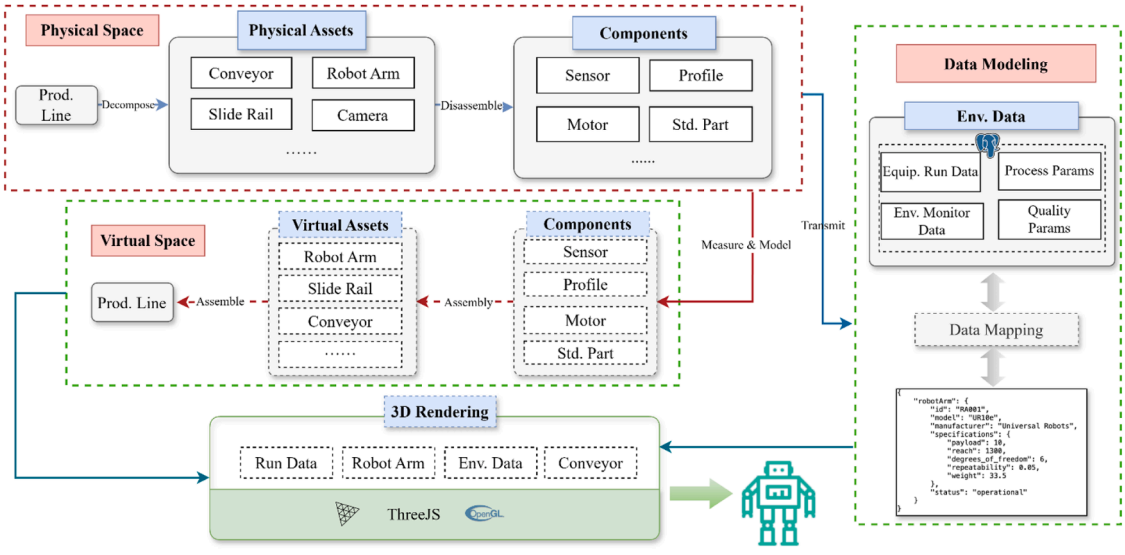


Fig. 8. Agentic digital twin asset binding.

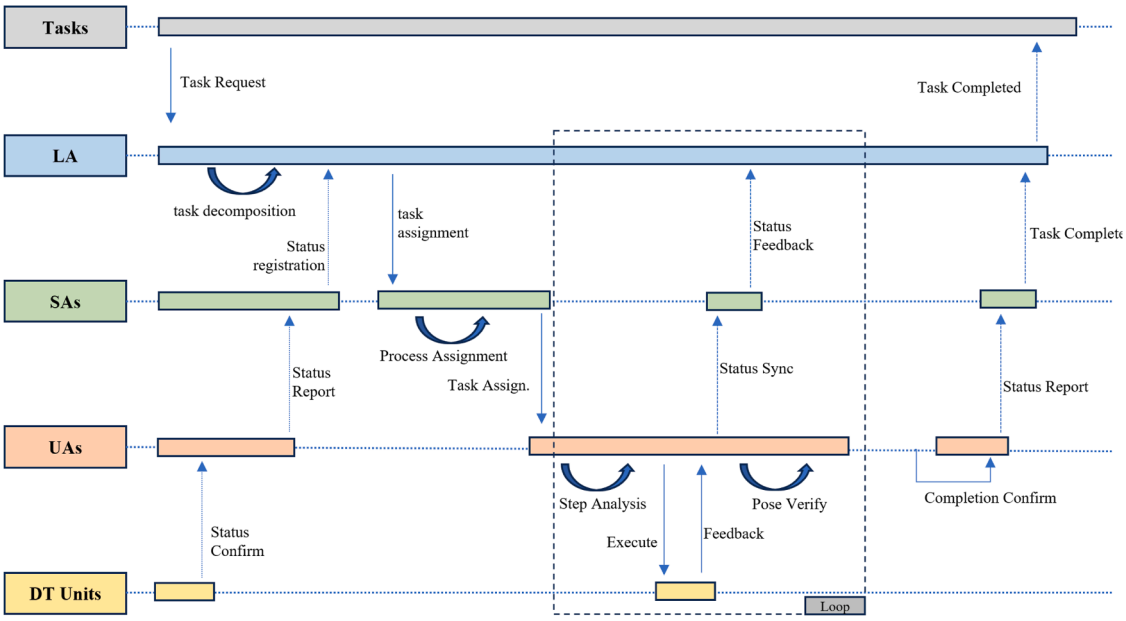


Fig. 9. Agent collaborative decision-making process.

serialized into a standardized JSON payload. Within this payload, the cognitive output is explicitly mapped to the instruction field as specific semantic directives or system commands. To ensure accurate execution by the physical assets, the language model also populates the parameters block with explicit geometric and operational data, such as target_pose coordinates and part_type identifiers.

The execution sequence is governed by protocol metadata, where the message_type designates the interaction context (such as task assignments, status reports, or knowledge queries) and the priority field dictates the execution order. Bottom-up feedback from the production line is encapsulated in the status_data field, delivering real-time sensor values and task completion metrics back to the language model. This continuous exchange translates complex cognitive tasks into machine-readable parameters and physical states into semantic context, closing the dynamic perception-action loop essential for the twin model's autonomous operation.

In the Task Decomposition (Top-Down) phase, the decision-making process initiates when the Line Agent receives a new production task T_{line}. The Agent acquires the current status of the production line based on the $M_{Knowledge}$ and executes the task decomposition operation f_{decom}, decomposing the macroscopic task into a sequence of module-level subtasks:

$$\{T_{mod1}, T_{mod2}, \ldots\ldots, T_{modn}\} = f_{decom}(T_{line}) \tag{21}$$

The corresponding Module Agent $Agent_{mod_i}$ performs a secondary decomposition to transform it into a finer-grained set of unit-level process instructions $T_{unit1}, T_{unit2}, \ldots, T_{unit_k}$ upon receiving the sub-task T_{mod_i}, and then issues them to the Unit Agents under its jurisdiction.

In the Status Feedback (Bottom-Up) phase, each Unit Agent $Agent_{unit_j}$, while executing tasks, continuously collects status information S_{unit_j} of its bound physical equipment (such as pose, temperature, machining results) and feeds this information back to its affiliated Module Agent $Agent_{mod_i}$ in real-time. The Module Agent aggregates the status of all units within its jurisdiction through the aggregation function f_{agg} to form a module-level status summary S_{mod_i}:

$$S_{mod_i} = f_{agg}(\{S_{unit_1}, S_{unit_2}, \ldots\ldots, S_{unit_n}\}) \tag{22}$$

Subsequently, the module level transmits the aggregated status information upwards to the Line Agent. The bottom-up data feedback closed-loop continuously refreshes the internal memory of agents at each hierarchical level, thereby ensuring that the top-level decision-maker can perceive the global task progress and the physical reality of the production line with real-time precision.

Through this continuous interaction driven by Top-Down and Bottom-Up, LLM-MAS dynamically weaves discrete instructions and status information into a $M_{behavior}$ capable of completely reproducing the operational logic of the production line. This model is not built once and for all but is continuously reconstructed and evolved during every task execution and status update, thereby achieving a precise simulation of the dynamic behavior of the flexible production line.

3.4. Multi-scale geometric evolution

$M_{geometry}$ acting as the definitive physical manifestation of knowledge and behavior, serves as a pivotal component in establishing a comprehensive closed-loop dynamic digital twin system. This model is responsible for precisely translating the scheduling and control instructions derived from $M_{behavior}$ into the evolution of geometric states and the reconstruction of topological structures within the twin space, thereby achieving a real-time mapping of the dynamic variations in the physical production line.

$M_{geometry}$ is not a static 3D scene but a dynamic evolving entity directly driven by $M_{behavior}$ and constrained by the semantic rules of the $M_{Knowledge}$. To address the diverse changes inherent in flexible production lines, an agent-driven, hierarchical $M_{geometry}$ dynamic update mechanism is introduced, which constructs a dynamic modeling paradigm driven by a Hierarchical Task Tree (HTT), as illustrated in Fig. 10. This paradigm explicitly models the dynamic task flow of the flexible production line as a tree-structured data structure, acting as a bridge connecting behavioral decision-making with geometric presentation. Furthermore, it decomposes model updates into two interrelated scales: state updates on the temporal scale, used to handle high-frequency dynamics during production line operation; and structural updates on the spatial scale, used to adapt to low-frequency reconfigurations of production line layouts and process flows.

3.4.1. Hierarchical task tree formulation

The Hierarchical Task Tree is a formalized, structured entity output by $M_{behavior}$ after decomposing and planning macroscopic production tasks. It transforms abstract collaborative processes into a computable

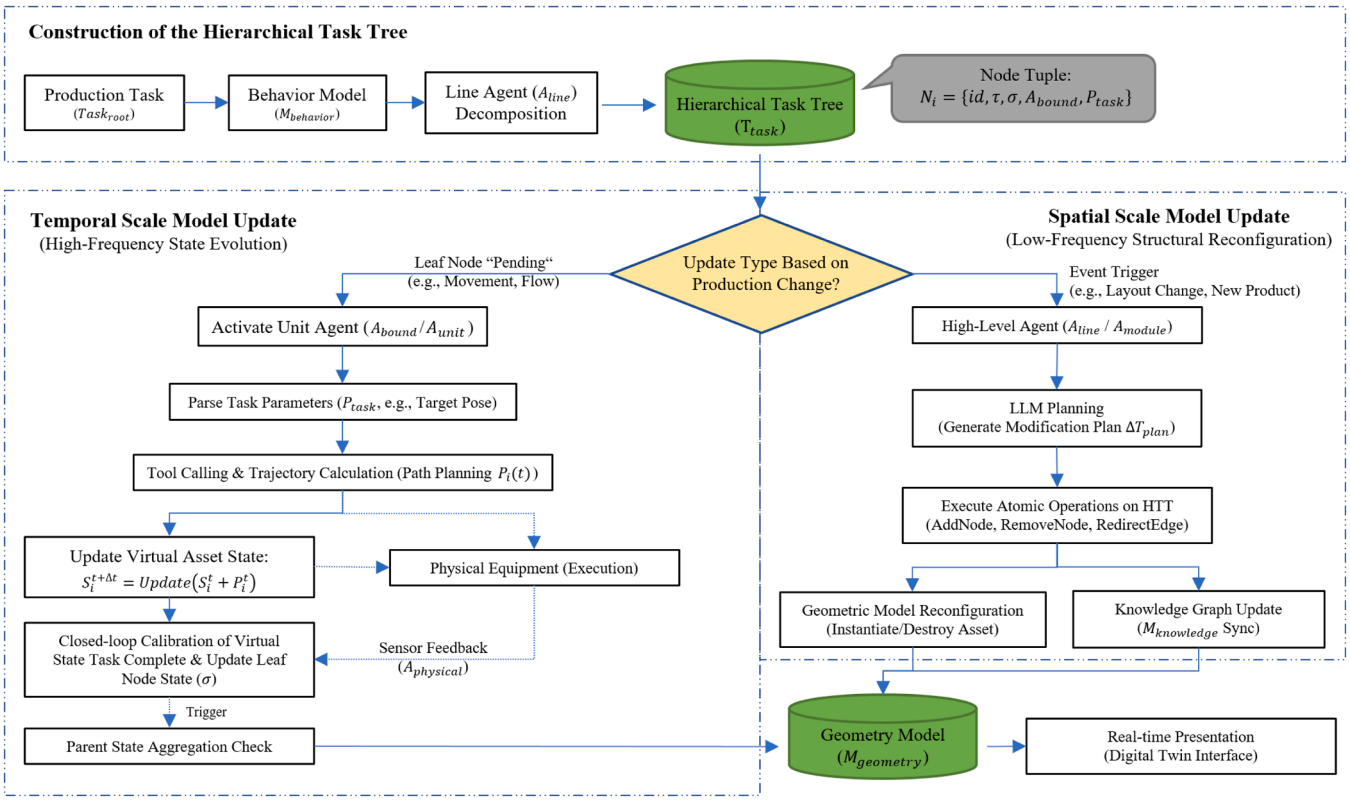


Fig. 10. Framework for HTT-driven multi-scale simulation.

and actionable data structure:

$$T_{task} = \left(\sum_{i=0}^{n} N_i + \sum_{i=0}^{n} E_i \right) \tag{23}$$

Where N is the set of nodes and E is the set of edges, representing the parent-child relationships between tasks. Each task node N_i is defined as a tuple:

$$N_i = (id, \tau, \sigma, A_{bound}, P_{task}) \tag{24}$$

Here, id is the unique identifier of the node; τ defines the node type, corresponding to line-level, module-level, and unit-level tasks respectively; σ represents the current lifecycle state of the node; A_{bound} is the agent bound to this node responsible for executing the task; and P_{task} is the set of parameters required to execute the task, such as target pose, machining parameters, etc. The construction process of the task tree can be expressed as a recursive decomposition function f_{decom} performed by the Line Agent A_{line} on the root Task $Task_{root}$:

$$T_{task} = f_{decom}(Task_{root}, A_{line}) \tag{25}$$

The tree-structured task tree constructed through this process constitutes the sole driving source for all subsequent geometric model updates. Agents continuously decompose tasks and construct dynamic task trees in response to different production tasks. This transforms discrete, event-driven production instructions from the physical production line into a logically clear, state-traceable digital object.

3.4.2. Temporal state synchronization

Updates at the temporal scale aim to synchronize high-frequency continuous dynamics during production operations in real-time, including robotic arm joint movements, AGV displacements, workpiece flow on conveyors, and equipment status indicator transitions. Characterized by the preservation of the production line's topological structure, these updates solely modify the position, pose, and appearance attributes of model elements. Within the HTT framework, this is defined as the execution process of a leaf node, which essentially functions as a state evolution function F_{time}.

When the state of a leaf node n_{leaf} becomes "pending", its bound Unit Agent $A_{bound(}n_{leaf})$ is activated. $M_{geometry}$ update driven by this agent can be expressed as:

$$M_{geometry}(t + \Delta t) = F_{time}\left(M_{geometry}(t), n_{leaf}\right) \tag{26}$$

Its execution flow is a closed-loop, continuous process. First, the bound Unit Agent A_{bound} parses the task goal, such as the target pose P_{target}, from the leaf node n_{leaf}. Subsequently, the agent utilizes Tool Calling to invoke path operation functions to calculate a trajectory $P_i(t)$ from the current pose $P_{current}$ to P_{target}, and drives the state S_i of the twin asset $Asset_i$ to update via API:

$$S_i^{t+\Delta t} = Update\left(S_i^t + P_i^t\right) \tag{27}$$

To ensure high fidelity, while updating the virtual model, instructions are also transmitted to the physical equipment. The agent performs closed-loop calibration of the virtual state S_i through continuous sensor feedback $S_{physical}$. Upon task completion, the agent updates the leaf node state. This state change automatically triggers a state aggregation check for the parent node (i.e., the parent-level agent), thereby achieving the autonomous advancement of the entire task flow.

3.4.3. Spatial structural reconfiguration

Spatial-scale updates focus on the reconfiguration of the production line's physical layout and topological structure triggered by production task variations (e.g., product changeovers) or process optimization. Such updates encompass structural evolution scenarios, including the addition of inspection stations, the replacement of robotic manipulators, and the global re-planning of AGV paths. In $M_{geometry}$, this is represented as a modification to the structure of the Hierarchical Task Tree T_{task} itself, which is a scene reconstruction function F_{space} triggered by an

event $E_{trigger}$.

$$T'_{task}, M'_{geometry} = F_{space}(T_{task}, M_{geometry}, E_{trigger}) \quad (28)$$

The core of this function lies in transforming scene reconstruction into a series of atomic operations on the task tree, triggering interlocking updates of $M_{geometry}$ and $M_{Knowledge}$. When a high-level agent (A_{line} or A_{module}) identifies an $E_{trigger}$, it utilizes the planning capabilities of the LLM to generate a modification plan ΔT_{plan} for the task tree. This plan contains a series of atomic structural modification operations, such as adding a node ($AddNode(n_{parent}, n_{new})$), removing a node ($RemoveNode(n_{target})$), or redirecting an edge (RedirectEdge(e_{old}, e_{new})). These operations further trigger chain reactions: adding a new task node calls the instantiation function of $M_{geometry}$:

$$M'_{geometry} = M_{geometry} \cup \{InstantiateAsset(n_{new}, P_{task})\} \quad (29)$$

Simultaneously, it updates the knowledge graph in $M_{knowledge}$, thereby achieving dynamic updating of production line knowledge. Conversely, deleting a node triggers the corresponding destruction of $M_{geometry}$ and updates to $M_{knowledge}$. This mechanism forms a direct causal chain: structural modifications to the $M_{behavior}$ automatically trigger structural reconstruction of the $M_{geometry}$, thereby achieving the adaptive evolution of the twin model.

3.4.4. Transactional consistency protocol

Considering potential issues such as agent response interruptions, planning errors, or conflicts caused by concurrent operations of multiple agents, a robust dynamic modeling model must possess state consistency and recoverability. To this end, a transactional update protocol to ensure the robustness of $M_{geometry}$ is introduced, depicted in Fig. 11. The core of this protocol is the combination of Hierarchical Lock and Transactional Update.

When structural modifications to the task tree on the spatial scale are being prepared by a high-level agent, an exclusive write lock must first be obtained on the root node of the target branch. This locking mechanism recursively propagates a frozen state to all descendant nodes, thereby pausing the task execution of their bound Unit Agents. This ensures isolation during reconstruction and avoids data races and logic conflicts. Once the lock is acquired, any spatial scale reconstruction is treated as an indivisible atomic transaction Γ.

This transaction can be formally expressed as an aggregate operation containing all state changes to the three core components: the task tree, the $M_{geometry}$, and the $M_{Knowledge}$. We denote the total changes for each component as ΔT_{task},$\Delta M_{geometry}$,$\Delta M_{knowledge}$, respectively. Therefore, the entire atomic transaction Γ is defined as an ordered tuple of these three change quantities:

$$\Gamma = (\Delta T_{task}, \Delta M_{geometry}, \Delta M_{knowledge}) \quad (30)$$

The system must ensure the atomicity of this transaction. To explicitly address the risks of agent failures, hallucinated outputs, or inconsistent logical decisions inherent in LLMs, the transactional update protocol serves as a critical fail-safe mechanism. Before any physical reconfiguration occurs, the generated modification plan (ΔT_{plan}) undergoes simulation verification. If the reconstruction plan generated by the LLM fails during the simulation verification phase (e.g., hallucinating an invalid coordinate or an illogical process sequence), or if an error occurs at any step of execution, the entire transaction will be rolled back as a whole. Consequently, this ensures that the task tree, $M_{geometry}$, and $M_{knowledge}$ are all restored to the stable state prior to the operation, discarding the erroneous agent output and preventing physical execution errors. The transaction is finally committed, and the lock released, only when all operations can be successfully executed.

4. Case study

4.1. Experimental setup

To evaluate the dynamic response capabilities of the proposed LLM-MAS, we conducted experiments on a parts defect monitoring and sorting line. This setup tests the ability of the framework to handle high part variety, complex inspection standards, and demands for high precision and speed. The simulation experiments emulate the operational status of the production line under complex scenarios involving diverse test subjects and process flow changes. The system monitors the physical production line in real time. It then dynamically adjusts the virtual model layout and state through agent negotiation to improve production efficiency and flexibility.

Fig. 12 illustrates the decoupled application architecture of the system. The ReactJS frontend provides visualization for the knowledge graph, virtual scenarios, and agent workflows. The Python and FastAPI backend manages data acquisition, agent management, and model

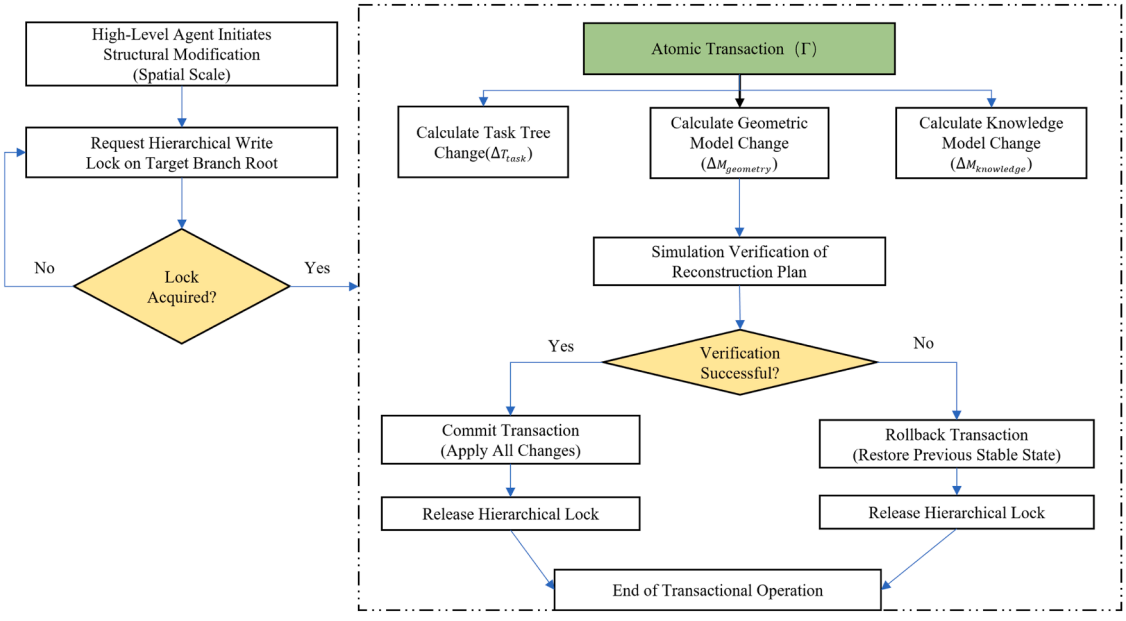


Fig. 11. Transactional update and rollback mechanism.

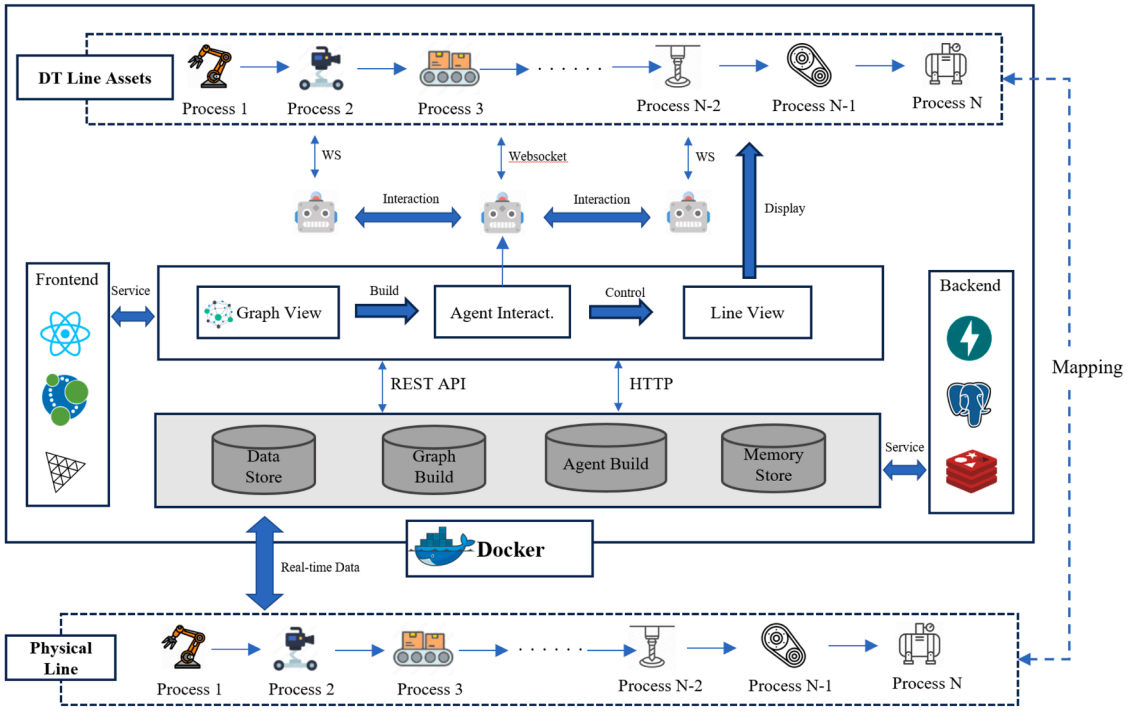


Fig. 12. Production line multi-agent interactive decision-making system.

maintenance. MySQL stores temporal operational data, and Neo4j manages the dynamic knowledge graph. Efficient real-time data transmission is supported by a multi-protocol communication architecture comprising RESTful APIs, HTTP, WebSockets, and shared memory message queues.

The experimental environment is deployed across a distributed hardware architecture. The host computer handles system scheduling and frontend rendering on a Windows 11 operating system with an Intel Core i7–12700H processor, 16 GB of RAM, and an NVIDIA RTX 3080 GPU. Core computational tasks and data processing are offloaded to a high-performance computation center running Linux 22.04.4 LTS, which is equipped with an 8-card NVIDIA A100 (40GB) GPU cluster.

The multi-agent cognitive core uses a heterogeneous LLM deployment strategy to balance reasoning capability, modality requirements, and system latency. The Line Agent and Module Agents use the DeepSeek-R1 model via API for global task decomposition, hierarchical planning, and logical reasoning. The vision-based Unit Agents use the GPT-4o model to process multimodal data, such as image-based defect recognition and spatial perception. The experimental parameter settings for both DeepSeek-R1 and GPT-4o include a temperature of 0.3, a top_p value of 1.0, and a single response limit. The max_tokens parameter is set to 64k, with streaming disabled. Both presence_penalty and frequency_penalty are set to 0. Inter-agent communication over WebSocket achieves an average semantic interaction latency of under 50 ms, ensuring high-fidelity temporal state synchronization across the digital twin system.

To maintain system stability over long-term operation and prevent context window overflow, the system dynamically flushes short-term interaction memory upon the closure of a production task. Persistent knowledge is offloaded to and retrieved from the Neo4j graph database, ensuring that memory growth does not degrade the LLM's reasoning efficiency or exceed the max_tokens processing limit.

4.2. Results and analysis

4.2.1. Feasibility validation

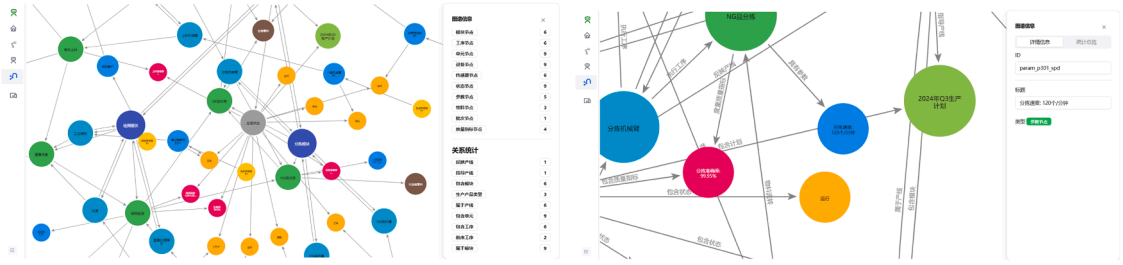
First, to construct domain-specific knowledge for the sorting line, the Graph Agent receives and processes CAD models containing the physical layout of equipment, as well as operation manuals and regulation documents that detail the features of parts to be sorted, defect standards, and sorting process logic. These multi-source heterogeneous data undergo preprocessing and segmentation before ingestion. Subsequently, the agent performs automated knowledge extraction based on the defined three-layer production line schema and finally stores the data in the Neo4j graph database.

The partial visualization of the production line knowledge graph is shown in Fig. 13(a) and 13(b). Statistical analysis was performed on the entities and relationships of the constructed knowledge graph to quantify its scale and complexity. The results show that the knowledge graph contains a total of 625 entity nodes, covering various production elements at the Line, Module, and Unit layers; and 867 relationship edges, effectively expressing the complex associations between elements at various levels of the production line. The specific statistics of entities and relationships are shown in Table 2.

To intuitively display the interaction workflow between agents, this study utilizes React Flow components to visualize the agent interaction workflow on the frontend platform and synchronously displays the natural language dialogue content between agents, as shown in Fig. 14. Furthermore, the platform implements real-time monitoring of individual production line modules, such as the real-time operational status of virtual assets like robotic arms and cameras.

In the figure, by monitoring the scenario of parts sorting task assignment in real-time, when a new parts sorting task is received, the Line Agent first perceives the task, decomposes it into specific sub-tasks, and assigns them to the subordinate Module Agents. The agent dialogue area on the right clearly records the task negotiation and decomposition process based on natural language.

The constructed platform can not only display the interactive workflow between agents but also dynamically update the digital twin production line model based on agent decisions. The platform constructs a 3D visualization interface of the digital twin production line via a 3D engine (WebGL). The real-time interaction between agents corresponding to each module is synchronously displayed on the right side of the interface. Through interaction, agents continuously alter the



(a). Partial schematic of the production line graph **(b).** Display of graph node knowledge

Fig. 13. Knowledge representation.

Table 2
Statistics of entity and relationship quantities.

Layer	Entity Name	Count	Relationship Name	Count
Line Layer	Production Line (PL)	1	ContainsModule	4
	Global Status (GS)	1	ReflectsProductionLine	1
	Product Type (PT)	5	Produce	5
Module Layer	Module	7	ContainsUnit	217
	Process	11	BelongsToProductionLine	7
	Material	5	PrecedesProcess	13
	Inspection Standard (IS)	5	RequiresMaterial	11
	Sorting Category (SC)	4	SuppliesMaterial	17
Unit Layer	Unit	217	BelongsToModule	213
	Process Unit (PU)	113	ExecutesStep	326
	Equipment	203	MonitorsEquipment	53
	Sensor	53		

production status of the production line, and these state changes are mapped in real-time to changes in the 3D model of the production line, as shown in Fig. 15.

The experiment verifies the system's dynamic response capability to process flow changes by executing sorting tasks for different types of parts. For example, when sorting hexagonal nuts, the agent system makes decisions based on the characteristics of the nuts and process requirements. The Unit Agent decides to adjust the motion trajectory of the robotic arm and the parameter configuration of the inspection module. These decision instructions are transmitted through agent interaction and displayed in real-time in the dialogue area on the right. Simultaneously, in the 3D interface on the left, the digital twin production line updates dynamically according to the agent's decisions; for instance, the pose and trajectory of the robotic arm change accordingly, and the position of the inspection unit may also be dynamically adjusted based on the part type. Through the real-time dynamic changes in the twin production line interface and the synchronous display of agent dialogue content, it is verified that the system can dynamically update the digital twin model based on agent decisions and achieve real-time linkage between the virtual production line and agent decisions. This indicates that the proposed system possesses the capability to dynamically adjust the production line model according to changes in production tasks, supporting flexible production operations.

4.2.2. *Comparative experiment*

To quantitatively evaluate and compare the efficiency and flexibility of Ordinary Flexible Production, Single-Agent Monitored Flexible Production, and Multi-Agent Collaborative Decision-Making Flexible Production within the parts defect monitoring and sorting line, specific Key

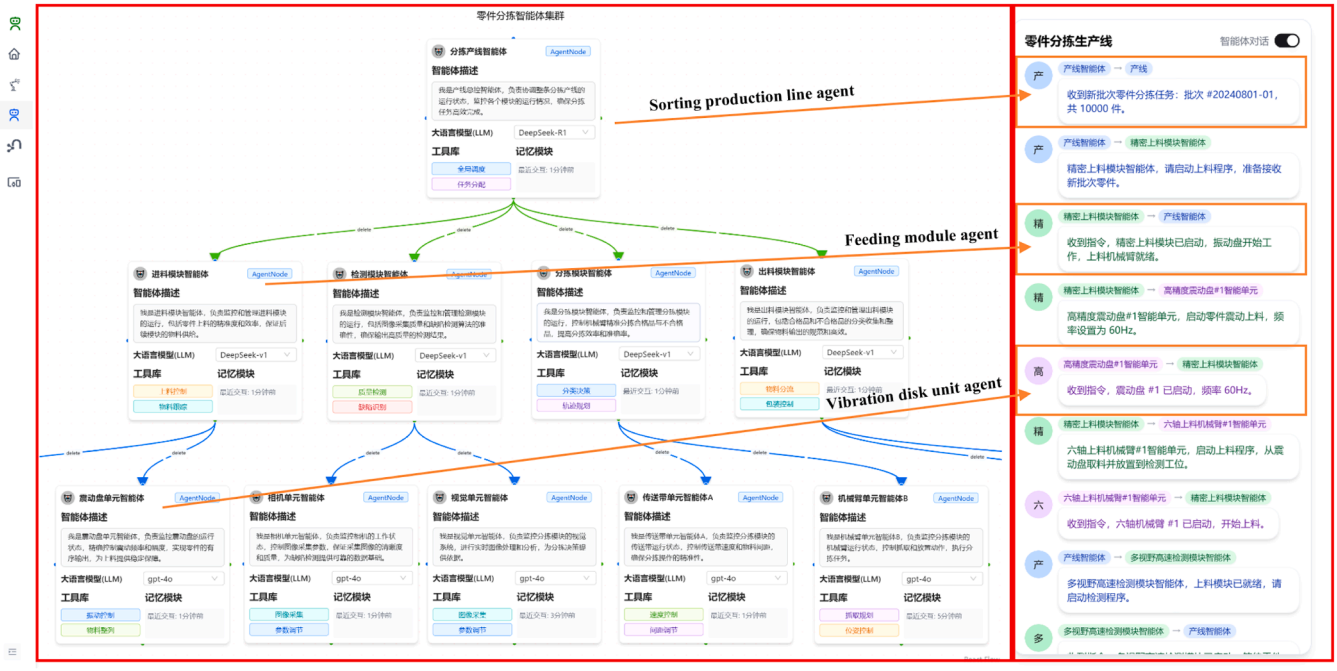


Fig. 14. Production line agent workflow.

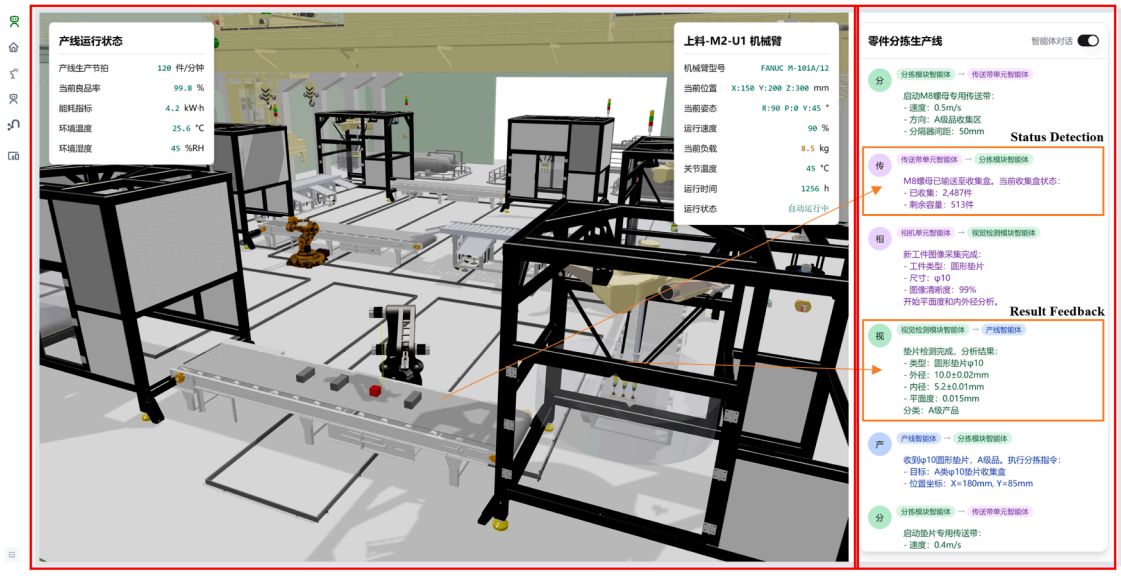


Fig. 15. Dynamic construction of the 3D production line model.

Performance Indicators (KPIs) were established.

In terms of efficiency, Throughput was selected to quantify the sorting output capacity per unit of time, while the False Positive Rate (FPR) was utilized to assess the extent of misclassification of qualified parts and the associated potential losses. Regarding flexibility, the focus was placed on the system's temporal response characteristics, specifically including: Changeover Time, which measures the efficiency of switching between different product types; Recovery Time from Disruption, which reflects the system's resilience in restoring normal operations following unexpected faults; and Process Adjustment Time, which evaluates the speed at which the system adapts to new process parameter configurations.

To comprehensively investigate the system's performance distinctions under different task loads, the production task involves sorting 100, 500, and 1000 parts respectively. Part types include hexagonal nuts, circular washers, screws, and rivets, with a defect mix ratio of 10%. Operational data for the three production modes on the parts defect monitoring and sorting line were tested under identical production tasks. The metric data for each production method according to different sorting quantities are shown in the tables below.

When the sorting quantity is 100 parts, the corresponding indicator data are Table 3.

When the sorting quantity is 500 parts, the corresponding indicator data are Table 4.

When the sorting quantity is 1000 parts, the corresponding indicator data are Table 5.

Table 3
Results for 100-part sorting.

Indicator	Unit	Ordinary Flexible Production	Single-Agent Production	Multi-Agent Collaborative Flexible Production
Throughput	parts/min	45.4	49.6	40.7
False Positive Rate	%	15.1	14.3	8.6
Changeover Time	min	20.4	18.7	10.0
Recovery Time	min	25.2	22.4	12.3
Process Adj. Time	min	20.8	18.3	12.9

Table 4
Results for 500-part sorting.

Indicator	Unit	Ordinary Flexible Production	Single-Agent Production	Multi-Agent Collaborative Flexible Production
Throughput	parts/min	50.1	46.7	36.8
False Positive Rate	%	17.5	16.1	9.4
Changeover Time	min	24.6	20.4	13.3
Recovery Time	min	25.7	20.3	11.4
Process Adj. Time	min	26.3	24.6	14.3

Table 5
Results for 1000-part sorting.

Indicator	Unit	Ordinary Flexible Production	Single-Agent Production	Multi-Agent Collaborative Flexible Production
Throughput	parts/min	52.3	44.5	34.6
False Positive Rate	%	18.1	17.7	9.8
Changeover Time	min	28.5	23.6	15.2
Recovery Time	min	27.3	21.5	11.1
Process Adj. Time	min	30.5	28.3	15.6

4.3. *Comparative performance evaluation*

The experimental results confirm the feasibility of the LLM-MAS in digital twin flexible production line applications. The system effectively constructs a production line knowledge graph consisting of 625 entity nodes and 867 relationship edges. It uses natural language interaction to achieve knowledge inquiry, task negotiation, and dynamic updating of the digital twin model, which lays the foundation for flexible production operations.

Efficiency validation reveals that the LLM-MAS performs well regarding the false positive rate. Even as the sorting task volume

increases to a high-load scenario, the system maintains low error levels of 8.6%, 9.4%, and 9.8% for the 100, 500, and 1000-part scenarios respectively. This performance is superior to ordinary flexible production, which exhibits a degrading trend from 15.1% to 18.1% in the 1000-part task. It also outperforms the single-agent scheme which records a 17.7% error rate at 1000 parts. This demonstrates the robustness of multi-agent collaboration in maintaining quality control under pressure.

Regarding flexibility indicators, the advantage of the LLM-MAS becomes increasingly pronounced as the scale of production expands. In the 1000-part high-load scenario, the LLM-MAS reduced changeover time from 28.5 min to 15.2 min and process adjustment time from 30.5 min to 15.6 min compared to ordinary flexible production. In terms of recovery time, traditional methods struggled to respond effectively under high loads and increased to 27.3 min, whereas the LLM-MAS maintained a rapid response capability of 11.1 min. This highlights its resilience against disruptions.

However, the throughput decreased from 40.7 to 34.6 parts per minute as volume increased, remaining lower than ordinary production which achieved 52.3 parts per minute at 1000 parts. This indicates the time overhead associated with frequent collaborative decision-making and communication between agents, which accumulates in large-scale tasks.

To analyze the source of these performance gains, it is necessary to analyze the relative contributions of the Knowledge-Behavior-Geometry (K-B-G) components. The behavioral coordination ($M_{behavior}$) contributes most significantly to the observed reductions in changeover and recovery times. By replacing rigid, hard-coded control logic with autonomous task decomposition and dynamic agent negotiation, the system eliminates the extensive manual reprogramming and human-in-the-loop latency typically required during product switchovers or fault handling. The knowledge evolution ($M_{Knowledge}$) serves as the critical enabler for these gains; its dynamic graph updates provide the real-time semantic context (e.g., immediate awareness of a new production plan or a failed robotic unit) that allows the agents to make instant, accurate decisions. Conversely, the geometry update ($M_{geometry}$) functions as the downstream execution mechanism. While it is strictly essential for maintaining the high-fidelity spatiotemporal synchronization and visual feedback of the twin, the actual time savings are primarily generated in the upstream cognitive and planning layers before being manifested geometrically.

The comparative experiments across the gradients of 100, 500, and 1000 parts reveal a clear trend. In small-batch tasks, the efficiency advantage of the LLM-MAS is not obvious due to collaboration overhead. As the task volume and complexity increase, its flexibility advantage emerges, particularly in recovery from disruptions. The LLM-MAS effectively mitigates the rapid degradation of quality and response speed seen in traditional methods under high loads. The limited increase in throughput and fluctuations in the false positive rate may be related to the inference latency and hallucination issues inherent in large language models. Although the system excels in intelligence and adaptability, future work is required to optimize communication protocols to reduce inference latency for large-scale industrial applications.

5. Conclusion

Addressing the intrinsic challenges of dynamic modeling for flexible production line digital twins, this paper proposes and validates a novel modeling paradigm driven by LLM-MAS. The study demonstrates that by defining the collaborative behavior of agents as the construction process of the twin model, this paradigm effectively addresses the limitations of traditional modeling methods regarding real-time responsiveness, adaptability, and semantic understanding, significantly enhancing the dynamic capabilities and intelligence level of digital twin systems. The main contributions of this paper include:

(1) Proposed a dynamic digital twin modeling paradigm. This paradigm transforms the digital twin model from a static, preset entity into a dynamic entity with autonomous evolutionary capabilities that emerges dynamically from multi-agent collaborative behaviors. This breaks through the limitations of traditional descriptive and prescriptive modeling.
(2) Established a Knowledge-Behavior-Geometry(K-B-G) ternary integrated digital twin model architecture. This architecture provides a semantic foundation via $M_{Knowledge}$, defines operational logic via $M_{behavior}$, and handles spatiotemporal presentation via $M_{geometry}$. It offers systematic theoretical guidance for the dynamic, multi-dimensional deconstruction and reconstruction of complex twin systems.
(3) Designed a semantic knowledge-driven closed-loop evolutionary mechanism for the twin model. This mechanism transforms environmental information into structured semantics through a dynamic Knowledge Graph and, via hierarchical task execution, operationalizes abstract knowledge into precise, atomic updates of the twin model's structure, behavior, and parameters, achieving true adaptability and self-consistency of the model.

The experimental results confirm that the proposed multi-agent framework significantly improves flexibility and fault tolerance in high-load manufacturing scenarios. The observed reductions in changeover and recovery times stem directly from the autonomous negotiation and dynamic task restructuring capabilities of the agents. Furthermore, the consistently low false positive rate highlights the reliability of combining semantic knowledge reasoning with real-time sensor data. We also documented a slight decrease in overall throughput compared to traditional rigid production lines. We interpret this as the inherent computational cost of continuous inter-agent communication and large language model inference during complex task execution. This specific trade-off between absolute processing speed and systemic adaptability suggests that our approach is well-suited for highly dynamic manufacturing environments where downtime and reconfiguration costs outweigh raw production speed. Future optimizations will focus on streamlining communication protocols and deploying edge-cloud computing to mitigate this inference latency.

Although this study has made certain progress in multi-agent digital twin system modeling, some limitations remain. The scale and complexity of the current validation scenarios have room for improvement, and future work needs to verify the system's robustness and scalability in broader industrial environments.

Furthermore, the performance of LLMs and the interaction overhead of MAS are key factors constraining the real-time performance of the system. While the Transactional Consistency Protocol effectively catches illogical outputs, the occasional hallucinations inherent in large language models can still trigger transaction rollbacks, leading to minor fluctuations in task throughput. Managing long-term agent memory growth without losing vital historical context also presents an ongoing challenge. Therefore, exploring model lightweighting, optimizing collaborative decision-making protocols, and integrating edge-cloud heterogeneous computing will be core research directions for advancing this paradigm toward industrial-grade applications.

CRediT authorship contribution statement

Ding Gao: Writing – original draft. **Xiaojian Wen:** Writing – review & editing. **Jianhao Lv:** Writing – review & editing. **Jiahui Si:** Writing – review & editing. **Cong Wang:** Writing – review & editing. **Jinsong Bao:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence

the work reported in this paper.

Data availability

The data that has been used is confidential.

References

[1] Y. Koren, U. Heisel, F. Jovane, T. Moriwaki, G. Pritschow, G. Ulsoy, H. Van Brussel, Reconfigurable manufacturing systems, CIRP Ann. 48 (2) (1999) 527–540. ISSN 0007-8506.

[2] J. Qin, Y. Liu, R. Grosvenor, A categorical framework of manufacturing for industry 4.0 and beyond, Procedia CIRP. 52 (2016) 173–178. ISSN 2212-8271.

[3] T. Tolio, D. Ceglarek, H.A. ElMaraghy, A. Fischer, S.J. Hu, L. Laperrière, S. T. Newman, J. Váncza, Design, management and control of flexible manufacturing systems, CIRP Ann. 59 (2) (2010) 777–801. ISSN 0007-8506.

[4] Grieves, M. Digital twin: manufacturing excellence through virtual factory replication. White paper, 2014.

[5] F. Tao, H. Zhang, A. Liu, A.Y.C. Nee, Digital twin in industry: state-of-the-art, IEEE Trans. Ind. Inform. 15 (4) (2019) 2405–2415. ISSN 1551-3203.

[6] Q. Qi, F. Tao, T. Hu, N. Anwer, A. Liu, Y. Wei, L. Wang, A.Y.C. Nee, Enabling technologies and tools for digital twin, J. Manuf. Syst. 58 (2021) 3–21. Part AISSN 0278-6125.

[7] M. Grieves, J. Vickers, Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. Transdisciplinary Perspectives on Complex Systems, Springer, Cham, 2017, pp. 85–113. ISBN 978-3-319-38756-7.

[8] Hongfei Guo, Minshi Chen, Khalgui Mohamed, Ting Qu, Siming Wang, Jianke Li, A digital twin-based flexible cellular manufacturing for optimization of air conditioner line, J. Manuf. Syst. 58 (2021) 65–78. Part BISSN 0278-6125.

[9] C. Zhuang, J. Liu, H. Xiong, Digital twin-based smart production management and control framework for the complex product assembly shop-floor, Int. J. Adv. Manuf. Technol. 96 (2018) 1149–1163. ISSN 0268-3768.

[10] A. Wortmann, B. Combemale, O. Barais, Self-adaptive manufacturing with digital twins, in: 2021 IEEE 18th International Conference on Software Architecture (ICSA), 2021, pp. 11–21. ISBN 978-1-6654-3444-5.

[11] M. Schluse, M. Priggemann, L. Atorf, J. Rossmann, Experimentable digital twins—Streamlining simulation-based development and operation of complex technical systems. Complex Systems Design & Management, Springer, Cham, 2018, pp. 315–326. ISBN 978-3-319-72145-7.

[12] P. Verma, M. Ajit, A semantic digital twin for autonomous systems, in: 2021 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), 2021, pp. 298–303. ISBN 978-1-6654-3923-5.

[13] T. Kojima, S.S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, Adv. Neural Inf. Process. Syst. 35 (2022) 22199–22213.

[14] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, J.R. Wen, A survey on large language model based autonomous agents, Front. Comput. Sci. 18 (6) (2024). Article 186345.

[15] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., He, X., Zhou, J., Wang, M., & Huang, F. The rise and potential of large language model based agents: a survey. arXiv preprint arXiv:2309.07864, 2023.

[16] Y. Lu, C. Liu, K.I.K. Wang, H. Huang, X. Xu, Digital Twin-driven smart manufacturing: connotation, reference model, and applications, Robot. Comput. Integr. Manuf. 61 (2020). Article 101837, ISSN 0736-5845.

[17] C. Zhuang, J. Liu, H. Xiong, A digital twin-based approach for data-driven cyber-physical systems, IEEe Access. 6 (2017) 6943–6951. ISSN 2169-3536.

[18] F. Tao, W. Liu, M. Zhang, T. Hu, Q. Qi, H. Zhang, F. Sui, T. Wang, H. Xu, A.Y. C. Nee, Five-dimension digital twin model and its ten applications, Comput.-Aided Des. 144 (2022). Article 103177ISSN 0010-4485.

[19] J. Leng, D. Wang, W. Shen, X. Li, Q. Liu, X. Chen, Digital twin-based manufacturing cyber-physical system for parallel controlling of smart workshop, J. Ambient. Intell. Humaniz. Comput. 10 (2019) 1155–1166. ISSN 1868-5137.

[20] T.H.J. Uhlemann, C. Lehmann, R. Steinhilper, The digital twin: realizing the cyber-physical production system for industry 4.0, Procedia CIRP. 61 (2017) 335–340. ISSN 2212-8271.

[21] C. Liu, P. Jiang, X. Zhang, A hypernetwork-based modeling and optimization of process planning for machining in a digital twin workshop, Robot. Comput. Integr. Manuf. 63 (2020). Article 101907ISSN 0736-5845.

[22] Y. Yi, Y. Yan, X. Liu, Z. Ni, J. Feng, J. Liu, Digital twin-based smart assembly process design and application framework for complex products and its case study, J. Manuf. Syst. 58 (2021) 94–107. Part AISSN 0278-6125.

[23] W. Kritzinger, M. Karner, G. Traar, J. Henjes, W. Sihn, Digital Twin in manufacturing: a categorical literature review and classification, IFAC-Pap. 51 (11) (2018) 1016–1022. ISSN 2405-8963.

[24] X. Xie, Z. Wang, Y. Chen, A four-dimension fusion model of digital twin and its application in the life-cycle management of satellite, IEEe Access. 8 (2020) 128501–128514. ISSN 2169-3536.

[25] S. Liu, J. Bao, Y. Lu, A multi-level data fusion framework for digital twin-based manufacturing systems, J. Manuf. Syst. 62 (2022) 540–552. ISSN 0278-6125.

[26] M. Wooldridge, An Introduction to Multiagent Systems, John Wiley & Sons, 2009. ISBN 978-0-470-51946-2.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 30, 2017.

[28] OpenAI. GPT-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[29] Anthropic. The Claude 3 model Family: opus, Sonnet, Haiku. Anthropic, 2024.

[30] P. Qian, et al., A survey on large language model based autonomous agents. arXiv preprint, 2023.

[31] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. ReAct: synergizing reasoning and acting in language models. arXiv preprint arXiv: 2210.03629, 2022.

[32] Shinn, N., Labash, B., & Gopinath, A. Reflexion: language agents with verbal reinforcement learning. arXiv preprint arXiv:2303.11366, 2023.

[33] G. Mialon, et al., Augmented language models: a survey, Trans. Mach. Learn. Res. (2023).

[34] Wang, J., et al. A survey on large language model based autonomous agents. arXiv preprint arXiv:2308.11432, 2023.

[35] Lee, J., & Su, H. (2023). A unified industrial large knowledge model framework in industry 4.0 and smart manufacturing. arXiv preprint arXiv:2312.14428.

[36] Liang, Jacky, et al. Code as policies: language model programs for embodied control. arXiv preprint arXiv:2209.07753 (2022).

[37] Ahn, Michael, et al. Do as i can, not as i say: grounding language in robotic affordances. arXiv preprint arXiv:2204.01691 (2022).

[38] J. Lee, H. Su, Agentic AI for smart manufacturing, Manuf. Lett. 46 (2025) 92–96. ISSN 2213-8463.

[39] Ren, Z., Zhang, Z., Luo, Z., & Li, X.L. Large language models for Industrial engineering: a survey. arXiv preprint arXiv:2402.13992, 2024.